AK-HPDST v21.0.0 Companion Documents

Problem-Interface Calibration, Execution, Reproducibility, and Release Handoff

Final Safety, Pipeline, and Release Re-audit

This final v21 re-audit projects the frozen mathematical architecture of the Main text, Foundation, DG, HT, UB, CM, TB, Technical Extension H–N, Problem Interface MS/NS, and Search/Platform U–Z into reusable calibration and execution contracts. It repairs strength-set, handoff-sort, external-input, migration, cache, replay, and release-state ambiguities without creating competing mathematical theorem authority. The complete v20 Companion corpus is preserved below as immutable predecessor ancestry.

Version 21.0.0

1 v21 Companion Constitution and Ownership Boundary

1.1 Role of the Companion layer

The v21 Companion layer is the operational projection of already owned mathematical and infrastructure results. Its purpose is to make source-specific calibration, route assembly, execution, storage, replay, audit, and release handoff explicit without converting a template, successful run, or complete package into a theorem.

$$\boxed{\text{Companion completeness} \;\not\Rightarrow\; \text{mathematical completeness}}$$

The canonical ownership boundary is:

(i) Main Chapters 2–6 and Foundation A–G own generic Core-Pmathematics;

(ii) Main Chapter 7 and Appendix DG own actual finite diagrams, natural transformations, coherence, and descent;

(iii) Main Chapter 8 owns proof-DAG and verifier handoff semantics;

(iv) Main Chapter 9 and Appendices HT/UB own generic Bridge composition and target-relative Return strength;

(v) MS and NS own source-specific Problem-Interface components, source-specific Bridge components, Return statements, and recovery lanes;

(vi) Appendices U–Z own agent, search, route, DAG, proof-store, execution, and semantic-replay infrastructure;

(vii) the Companion owns operational templates, calibration instances, execution envelopes, and cross-document handoff records only.

Definition 1.1 (C-v21-DEF-01: Companion projection)**.** A *Companion projection* is a typed, target-specific map from a canonical theorem, interface, or infrastructure record to an operational schema. It records the source owner, exact theorem or contract identity, source and target schemas, consumed fields, preserved fields, discarded advisory fields, hypotheses, artifact identities, and consumer ceiling.

Proposition 1.2 (C-v21-DER-01: Projection is non-promotional)**.** *A Companion projection does not strengthen its source result. Every operational conclusion is bounded above by the exact theorem, interface, or infrastructure strength named in the projection record.*

Proof. The projection supplies representation and workflow data only. It neither weakens a missing hypothesis nor creates an implication absent from the source owner. Therefore its consumer ceiling is inherited from the source record. □

Definition 1.3 (C-v21-DEF-02: Independent status and result sorts)**.** The Companion maintains the following pairwise distinct fields.

(a) atomic verifier status:

$$\text{pass}, \quad \text{reject}, \quad \text{undefined}, \quad \text{not\_invoked};$$

(b) P2DG workflow readiness:

$$\text{ready}, \quad \text{incomplete}, \quad \text{inconsistent}, \quad \text{not\_invoked};$$

(c) the positive handoff tokens descent_input_ready and bridge_input_ready, emitted only after their own prerequisite boundaries pass; absence of a token is not itself a negative status and is explained by the constituent atomic statuses and scope record;

(d) Type IV diagnostic status:

$$\text{not\_invoked}, \quad \text{undefined}, \quad \text{certified\_zero}, \quad \text{obstructed};$$

(e) for one fixed target, the Bridge-result sort:

$$\text{reject}, \quad \text{undefined}, \quad \text{obstructed}, \quad \text{partial\_return}, \quad \text{regularized\_return}, \quad \text{return};$$

(f) calibration, execution, storage, cache, replay, migration, audit, Claim-Register, package-release, publication, and registration lifecycles.

Lexical overlap never identifies two fields. No value is copied between sorts, and a handoff token is never serialized as a theorem result.

Proposition 1.4 (C-v21-DER-02: Status-axis noninterference)**.** *Changing a calibration, storage, execution, replay, or audit status does not change a mathematical verifier or Bridge result unless a separately declared verifier-side conversion is executed and passes.*

Proof. The status fields inhabit distinct typed codomains and answer different predicates. A conversion between them is an additional operation with its own proof obligations. □

Definition 1.5 (C-v21-DEF-03: Lossless Companion packet)**.** A v21 Companion packet is

$$\begin{aligned}\mathsf{CalibrationPacket} = \big(&\text{id}, \text{kind}, \text{owner}, C_{\text{src}}, C_{\text{tgt}}, \text{variance}, \text{coeff/endpt},\\ &\text{scope}, \text{predicate}, \text{Hyp}, \text{mode}, \text{stage}, \text{status}, \text{readiness}, \text{result},\\ &\Gamma, \mathfrak{S}, \text{Fail}, \text{authority}, \text{environment}, \text{provenance}, \mathcal{A}\big).\end{aligned}$$

The packet is reusable only when the consumer accepts every consumed field at the exact recorded type and strength.

Definition 1.6 (C-v21-DEF-04: Consumer projection)**.** For a consumer K, the *consumer projection* $\pi_K(P)$ is the ordered collection of packet fields actually consumed by K, together with the consumer's acceptance predicates. Fields not read by K are advisory relative to that consumer and cannot influence its result.

Proposition 1.7 (C-v21-DER-03: Consumer-projection invariance)**.** *Let packets P and Q satisfy $\pi_K(P) = \pi_K(Q)$, including identical theorem, source, environment, and artifact identities. If K is deterministic and has no undeclared side effects, then $K(P) = K(Q)$.*

Proof. The consumer receives identical effective input and pinned environment. Determinism then gives identical output. □

Definition 1.8 (C-v21-DEF-05: Semantic field map)**.** A packet adapter $A : P \to Q$ carries a partial typed semantic field map

$$\sigma_A : \mathrm{Dom}(\sigma_A) \subseteq \mathcal{F}(P) \longrightarrow \mathcal{F}(Q).$$

For composable adapters A and B, the composite map is defined precisely on fields for which both stages are defined, and equals $\sigma_B \circ \sigma_A$ there.

Proposition 1.9 (C-v21-DER-04: Preservation certificates compose)**.** *Transformation-preservation certificates compose along the semantic field maps of Definition 1.8. A field rename, schema migration, translation, or serialization is therefore safe only on the domain explicitly carried through the composite map.*

Proof. Every end-to-end preserved field has a defined image at each intermediate adapter and each local preservation certificate applies. A field outside that domain lacks an end-to-end preservation proof. □

Definition 1.10 (C-v21-DEF-06: Compatible packet amalgamation)**.** A finite family of packets is *jointly compatible* when every shared semantic field has the same value, theorem identity, source binding, status sort, and artifact identity, and the union of hypotheses is consistent. Its amalgamation is the union packet $\bigsqcup_i P_i$. Incompatible families yield undefined; no priority, latest-wins, or silent override rule is used.

Proposition 1.11 (C-v21-DER-05: Algebra of compatible amalgamation)**.** *On jointly compatible packets, amalgamation is commutative, associative, and idempotent.*

Proof. The amalgamation is the unique union of mutually equal shared fields and disjoint nonshared fields. Set-theoretic union has the stated properties on this domain. □

Definition 1.12 (C-v21-DEF-07: Target-relative evidence support profile)**.** For every declared target T, a Companion object carries a downward-closed supported-strength set

$$\mathfrak{S}_T \subseteq \mathrm{Strength}(T)$$

licensed by the named source theorems and complete routes. The preorder may have incomparable maximal elements and need not have a strongest member. A finite presentation may record the Pareto frontier of maximal supported strengths. A template, execution, replay, audit, or release event cannot enlarge $\mathfrak{S}_T$.

Proposition 1.13 (C-v21-DER-12: Alternate-route support union)**.** *Let complete alternative routes to the same target T support downward-closed sets $\mathfrak{S}_1, \ldots, \mathfrak{S}_m$. The route portfolio supports*

$$\mathrm{Down}\left(\bigcup_{j=1}^{m} \mathfrak{S}_j\right).$$

An incomplete route contributes no strength, and components from distinct incomplete routes are not spliced unless a new theorem-backed route explicitly composes them. For independent targets in categories $C_1, \ldots, C_m$, the combined record lies in $\prod_j C_j$ and its supported set is the Cartesian product $\prod_j \mathfrak{S}_j$, not an untyped intersection.

Proof. A complete route proves each strength in its own supported lower set, so the portfolio proves the downward closure of their union. Incomplete paths prove no route conclusion. Independent targets are combined by the categorical product record and retain coordinatewise strengths. □

Proposition 1.14 (C-v21-DER-06: Infrastructure noncompensation)**.** *Pipeline width, route multiplicity, storage redundancy, cache hits, replay success, audit completion, and release packaging cannot compensate for a missing mathematical edge, source authority, actual morphism, descent theorem, Bridge component, Return theorem, or target-specific verifier.*

Proof. The listed infrastructure predicates are not proofs of the missing mathematical predicates and inhabit separate status sorts. □

1.2 Route portfolios and target-local operation

Definition 1.15 (C-v21-DEF-08: Companion route portfolio)**.** A *Companion route portfolio* is a finite typed DAG of source bindings, adapters, calibration packets, Core or DG certificates, source-specific Bridge components, Return records, execution nodes, and target consumers. A selected route is complete only when every required node and edge in its target slice is current and discharged, or is explicitly outside scope through an immutable exclusion record. Optional unused branches do not enter that route.

Definition 1.16 (C-v21-DEF-09: Target slice and target cut)**.** For target t, the target slice $D_{\leq t}$ is the induced sub-DAG on all ancestors of t and t. A target cut is any node set meeting every complete source-to-t route. No antichain property is assumed for a general inclusion-minimal cut.

Proposition 1.17 (C-v21-DER-07: Target-slice sufficiency)**.** *Assume a target slice is dependency-complete, all executable nodes are deterministic functions of declared predecessors and pinned environments, and no undeclared global side effects occur. Then evaluating the slice in a topological order is sufficient to determine the target. Nodes outside the slice are irrelevant to that target.*

Proof. Every consumed influence is represented inside the slice by dependency completeness. Topological evaluation supplies every predecessor before its consumer. Outside nodes have no path into the target. □

Proposition 1.18 (C-v21-DER-08: Complete fallback and failure isolation)**.** *A failed route does not invalidate an alternative route to the same declared target whose complete target slice avoids the failed node, whose shared ancestors remain current, whose assumption context is satisfied, and whose portfolio-wide invariants remain valid. The alternative exports only its own supported-strength set. Pieces of distinct incomplete routes may not be combined unless a new theorem-backed route explicitly composes them.*

Proof. The alternative route retains every required premise. Cross-route borrowing would create a path absent from the declared route DAG and is therefore unsupported. □

Definition 1.19 (C-v21-DEF-10: Adaptive validation firewall)**.** A theorem-facing validation freezes the candidate, target predicate, validation corpus, scoring rule, stopping rule, and disclosure policy before validation. If validation outcomes are inspected and the candidate is revised, the validation corpus becomes search input; a new holdout or an adaptive-validity theorem is required.

Proposition 1.20 (C-v21-DER-09: Validation reuse boundary)**.** *The same data may support both search and independent validation only when an explicit theorem accounts for the adaptive dependence. Mere nondisclosure language or agent separation does not establish independence.*

Proof. Independence is a property of the dependency graph and information flow, not of labels or personnel alone. □

Definition 1.21 (C-v21-DEF-13: Legacy-schema current-reading map)**.** The frozen v20 Companion tokens are interpreted in a v21 run only through a field-qualified current-reading map.

(a) checked, verified, complete, adequate, replay-passed, audit-passed, and R0–R5 remain lifecycle, support, or reproducibility values in their original fields.

(b) not_applicable and not_used map to not_invoked only through an immutable scope-exclusion record; otherwise theorem-facing use is undefined.

(c) a legacy Bridge field such as candidate, [Spec], theorem, or rejected is split into the current document role, evidence status, invocation role, atomic obligations, and target-indexed Bridge result. The word theorem is not accepted without its exact source binding and proof identity.

(d) legacy combined fields are projected into the current comparison mode, detector stage, readiness, result, workflow, and artifact fields without copying a favorable token across sorts.

Proposition 1.22 (C-v21-DER-13: Legacy favorable tokens are nonpromotional)**.** *No favorable v20 template token determines a v21 verifier pass, handoff token, Bridge result, Claim-Register promotion, package release, or publication status without the corresponding current-reading conversion and complete target route.*

Proof. The predecessor token answers its historical field predicate. Every listed v21 value belongs to a distinct field and requires its own current evidence and authority. □

2 v21 Companion 1: Problem-Interface Calibration

2.1 Calibration bus and source-specific lanes

The v20 arithmetic examples are retained below. The v21 Companion broadens the operational role to cover the MS arithmetic lanes and NS PDE lanes through one typed calibration bus, while keeping their mathematics and Return statements source-specific.

Definition 2.1 (C1-v21-DEF-01: Problem-Interface calibration packet)**.** A Problem-Interface calibration packet is a PIPacket-compatible CalibrationPacketthat identifies one external source class, one source adapter, one exact AK-side predicate, one source-specific Bridge component or Return target, and the complete source-binding and artifact package required by that lane.

Definition 2.2 (C1-v21-DEF-02: Source-adapter registry)**.** The source-adapter registry records typed adapters from arithmetic, analytic, computational, or proof-assistant source data into calibration packets. Each adapter declares source semantics, variance, coefficients, endpoint conventions, loss profile, preservation theorem, and consumer ceiling.

Proposition 2.3 (C1-v21-DER-01: Conservative adapter extension)**.** *A version-pinned source adapter in an isolated namespace widens calibration capability without changing existing route conclusions. This requires that no mutable alias shadow an existing adapter and that every existing packet and theorem identity remain unchanged.*

Proof. Every old target slice and name resolution remains unchanged. New capability appears only through new typed edges. □

Definition 2.4 (C1-v21-DEF-03: MS arithmetic lane record)**.** An MS arithmetic lane record declares exactly one of the following source-specific families:

(a) finite-length DVR module or finite Abelian p-group realization;

(b) Iwasawa growth, recurrence, residual, or rational generating-function packet;

(c) strict bounded-control diagram portfolio;

(d) congruence, character, residue-class, or signed branch decomposition;

(e) source-bound Known-Theorem Recovery or conditional specialization.

The record states whether its output is an exact invariant, a conditional Bridge component, a Return candidate, an advisory signal, or an unresolved program.

Definition 2.5 (C1-v21-DEF-04: NS analytic lane record)**.** An NS analytic lane record declares the solution class, space-time domain, exponent pair (q, r), scaling index, local or global norm packet, approximation topology, local-cover or time-partition data, source regularity criterion, and exact Return direction. Criterion-level recovery and unconditional regularity are separate targets.

Proposition 2.6 (C1-v21-DER-02: Source-specific lane noninterchangeability)**.** *An MS packet and an NS packet may share infrastructure fields, but neither may discharge a source-specific theorem obligation of the other. Shared packet syntax does not identify arithmetic and analytic predicates.*

Proof. The source categories, semantics, hypotheses, and Return predicates differ. Infrastructure compatibility does not supply a cross-domain theorem. □

Definition 2.7 (C1-v21-DEF-05: Calibration target portfolio)**.** A calibration target portfolio records one or more independently declared targets, such as invariant extraction, Core-P evaluation, Core-DG diagram data, Bridge readiness, source-specific Return, regression validation, or formalization. Each target has its own accepted packet projection and downward-closed supported-strength set.

Proposition 2.8 (C1-v21-DER-03: Immutable calibration fan-out)**.** *One immutable calibration packet may feed multiple consumers. If consumer i has target category C_i and supported lower set $\mathfrak{S}_i$, the successful combined record lies in $\prod_i C_i$ with supported set $\prod_i \mathfrak{S}_i$. Failure of one coordinate does not invalidate another unless the failed predicate is an explicit dependency of that coordinate. No cross-target equation, diagonal identification, or common-target Return follows without an additional theorem.*

Proof. Each consumer reads its own projection and produces its own target-relative record. Independent records combine coordinatewise in the product category. □

2.2 Arithmetic calibration reinforcement

Definition 2.9 (C1-v21-DEF-06: Finite-length module calibration packet)**.** A finite-length module packet records a DVR (R, π, k), a finite-length module M, an elementary-divisor or Hilbert-profile source, the length-persistence realization, coefficient conventions, and the exact invariant target to be recovered. Complete module reconstruction is consumed only through the source-specific MS theorem that identifies the barcode with the elementary divisors.

Proposition 2.10 (C1-v21-DER-04: Equivalent arithmetic calibrations)**.** *Within the hypotheses of the registered MS theorem, the elementary-divisor multiset, the Hilbert profile together with its finite support, and the complete length-barcode are mutually convertible calibration representations. This equivalence does not recover birth positions of an unrelated persistence object or a stronger arithmetic theorem.*

Proof. The conversions are exactly those supplied by the registered finite-length module result. The stated ceiling excludes data not encoded by that theorem. □

Definition 2.11 (C1-v21-DEF-07: Recurrence calibration packet)**.** A recurrence packet binds a sequence, shift operator, annihilating polynomial, tail start, residual sequence, initial values, exact arithmetic domain, and proof of the recurrence. Observed values without the recurrence proof remain samples rather than a tail certificate.

Proposition 2.12 (C1-v21-DER-05: Certified finite tail replay)**.** *For a proved monic order-d recurrence valid from N, the recurrence packet and the d exact values $c_N, \ldots, c_{N+d-1}$ determine the tail uniquely. Exact replay is licensed only in a backend preserving the coefficient domain, indexing, recurrence operator, exact arithmetic, and canonical serialization; floating-point rounding, overflow, or an unproved coefficient conversion gives at most a separately certified approximation.*

Proof. A monic recurrence recursively determines every subsequent term in the declared coefficient domain. Exact backend substitution requires preservation of every operation and representation consumed by that recursion. □

Definition 2.13 (C1-v21-DEF-08: Branch calibration packet)**.** A branch packet records a finite residue, character, parity, signed, or local-component decomposition, a complete branch cover, branch-specific source bindings, and the theorem that recombines the branch outputs. Missing branches are undefined, not zero.

Proposition 2.14 (C1-v21-DER-06: Branch portfolio completeness)**.** *A branch portfolio supports a global target only when the branch family is proved complete and the recombination theorem applies. Agreement on the inspected branches alone does not prove the global target.*

Proof. Uninspected or unbound branches may carry additional data. Completeness and recombination are independent obligations. □

2.3 PDE calibration reinforcement

Definition 2.15 (C1-v21-DEF-09: Mixed-norm certificate packet)**.** A mixed-norm certificate packet records (q, r), the exact Bochner space, temporal and spatial scopes, local certificate family, aggregation rule, scaling reserve, approximation mode, and source criterion. A displayed finite norm is theorem-facing only when the packet is source-bound and verified.

Definition 2.16 (C1-v21-DEF-10: Local cylinder portfolio)**.** A local cylinder portfolio consists of a finite selected family of parabolic cylinders, dimensionless local packets, a declared coverage or overlap rule, and a source theorem whose hypotheses are verified on each consumed cylinder. A countable or continuum family enters one run only through a separately certified compactness, finite-subcover, well-founded, or finite-to-infinite reduction. The portfolio does not create an epsilon-regularity theorem.

Proposition 2.17 (C1-v21-DER-07: Local-to-global certificate assembly)**.** *When a registered NS theorem supplies the required finite-cover or time-partition aggregation inequality, verified local packets may be assembled into the corresponding global norm certificate. The assembly inherits the exact exponent, domain, and solution-class restrictions of that theorem.*

Proof. The global bound is the conclusion of the registered aggregation theorem applied to the verified local inputs. No stronger criterion follows. □

Definition 2.18 (C1-v21-DEF-11: Approximation-limit packet)**.** An approximation-limit packet records the approximate solution family, uniform norm bound, exact weak or strong topology, extracted limit, source-solution identification, and lower-semicontinuity or compactness theorem used to transport the bound.

Proposition 2.19 (C1-v21-DER-08: Limit transport is topology-scoped)**.** *A bound is transported from approximants to a limit only in the topology and function space covered by the registered theorem. Numerical convergence, convergence in a weaker topology, or convergence of selected observables is insufficient unless a separate theorem connects it to the consumed norm.*

Proof. Lower semicontinuity and compactness are topology-specific. A different convergence statement does not instantiate their hypotheses. □

Definition 2.20 (C1-v21-DEF-12: Heterogeneous criterion stitching record)**.** A heterogeneous criterion stitching record partitions time into finitely many intervals, names one source-bound regularity criterion per interval, records the verified hypotheses and result on each interval, and binds compatible endpoint traces or overlap data. It does not require a common exponent pair unless the stitching theorem does.

Proposition 2.21 (C1-v21-DER-09: Criterion stitching ceiling)**.** *A heterogeneous stitching record yields at most the global regularity statement proved by the registered stitching theorem. It does not prove that any one criterion holds globally and does not eliminate an interval with missing evidence.*

Proof. The global conclusion is obtained by composing the interval conclusions and compatibility data. A missing interval breaks the composition. □

Definition 2.22 (C1-v21-DEF-13: Calibration-family coverage certificate)**.** A calibration-family coverage certificate records the exact domain covered by a finite packet family and the theorem that promotes that finite family to the requested global, tail, or continuum scope. The certificate distinguishes a finite subcover, finite partition, finite-state reduction, compactness argument, and source-specific finite-to-infinite theorem.

Proposition 2.23 (C1-v21-DER-10: No implicit continuum coverage)**.** *A finite collection of arithmetic stages, time intervals, spatial sets, or parabolic cylinders proves statements only on its declared union. A larger or infinite target scope requires the coverage certificate of Definition 2.22; finiteness of the record alone is not completeness.*

Proof. Points, stages, or branches outside the declared finite union are not constrained by the local records. Only the named coverage or reduction theorem supplies the missing implication. □

2.4 Calibration schema

```
problem_interface_calibration:
  id: "cal-packet-0001"
  version: "21.0.0"
  owner: "MS / NS / other source-specific owner"
  source_binding:
    artifact_id: "content-addressed source binding"
    theorem_locator: "exact locator"
    permitted_use: "declared"
    prohibited_use: "declared"
  adapter:
    id: "version-pinned adapter"
    source_schema: "declared"
    target_schema: "PIPacket / CalibrationPacket"
    semantic_field_map: "artifact reference"
  lane:
    family: "DVR-module / Iwasawa / control / mixed-norm / local-cylinder / approximation"
    target: "exact target predicate"
  route:
    target_slice: "route-slice artifact"
    fallback_routes: []
  status_axes:
    atomic_verifier_status: "pass / reject / undefined / not_invoked"
    p2dg_readiness: "ready / incomplete / inconsistent / not_invoked"
    descent_handoff:
      token: "descent_input_ready / absent"
      prerequisite_status: "pass / reject / undefined / not_invoked"
    bridge_handoff:
      token: "bridge_input_ready / absent"
      prerequisite_status: "pass / reject / undefined / not_invoked"
    bridge_result: "reject / undefined / obstructed / partial_return / regularized_return / return"
    lifecycle: "draft / checked / frozen / superseded"
  supported_strengths:
    representation: "downward-closed set / finite Pareto frontier"
  artifacts: []
  non_claims:
    - "Calibration does not create a source theorem."
    - "Bridge readiness is not Return."
```

Listing 1: v21 Problem-Interface calibration packet

3 v21 Companion 2: Execution, Reproducibility, and Audit

3.1 Hermetic execution and proof closure

Definition 3.1 (C2-v21-DEF-01: Hermetic execution envelope)**.** A hermetic execution envelope pins code, effective inputs, target slice, operating system and architecture when relevant, locale and time-zone rules, environment, backend, canonicalization, toolchain, external responses promoted to immutable inputs, resource limits, randomness policy, output schema, and failure aggregation. Undeclared clocks, mutable globals, network calls, hardware-dependent reductions, and registry resolution are forbidden unless their semantics are explicitly captured by the envelope.

Definition 3.2 (C2-v21-DEF-02: Execution packet)**.** An execution packet is an ExecutionPacket-compatible projection of a calibration or proof packet containing the target slice, command graph, artifact closure, environment, cache policy, verification modes, replay criteria, and target-indexed downward-closed supported-strength sets.

Proposition 3.3 (C2-v21-DER-01: Hermetic determinism)**.** *Two runs with identical complete hermetic envelopes and deterministic node semantics produce equal canonical outputs or the same canonical failure record under the declared output equivalence. Byte identity follows only when canonical byte serialization is itself part of the pinned envelope.*

Proof. The complete effective input and every permitted execution choice are fixed. Determinism fixes the semantic result, and pinned canonical serialization fixes its bytes when byte equality is requested. □

Definition 3.4 (C2-v21-DEF-03: Merkle proof closure manifest)**.** A Merkle proof closure manifest canonically serializes each object's payload and ordered dependency identifiers, together with a canonically ordered root set, hash algorithm, domain-separation rules, encoding version, and scheme specifications. Injectivity is required only on the finite release closure; no global cryptographic collision theorem is asserted.

Proposition 3.5 (C2-v21-DER-02: Closure commitment)**.** *Under canonical serialization and collision-freeness on the finite release closure, equal Merkle root identifiers with equal scheme specifications imply byte-identical rooted dependency closures and dependency incidence. They do not imply semantic equivalence or mathematical validity.*

Proof. The recursive commitment determines every canonical payload and ordered dependency list in the finite closure. Meaning and truth are predicates external to byte identity. □

Definition 3.6 (C2-v21-DEF-04: Cache-admission certificate)**.** A cache-admission certificate binds a cache key to the complete effective input and transitive dependency closure, environment, code, backend, canonicalization, output identity, and verifier mode. It validates the output by trusted-closure membership, proof-carrying verification, independent extensional checking, or an equivalent declared method before theorem-facing consumption. A remote key match or signature alone is insufficient.

Proposition 3.7 (C2-v21-DER-03: Verified cache substitution)**.** *A cache object admitted by a valid cache-admission certificate may replace recomputation for deterministic downstream consumers whose projections are contained in the certificate. Consumers reading additional fields require a new admission check.*

Proof. The admitted object is extensionally equal on every field consumed by the stated downstream consumers. Additional fields lie outside the proved domain. □

3.2 Incremental and parallel execution

Definition 3.8 (C2-v21-DEF-05: Structural stale set and rebuild plan)**.** Partition nodes into immutable identity/input nodes V_{id} and executable nodes V_{exec}. For changed nodes $C \subseteq V_{\mathrm{id}} \cup V_{\mathrm{exec}}$, let $\mathrm{Desc}^*(C)$ denote the reflexive descendant closure. The structurally stale executable set is

$$\mathrm{Stale}_{\mathrm{exec}}(C) = \mathrm{Desc}^*(C) \cap V_{\mathrm{exec}}.$$

For targets T, the default-safe rebuild plan is

$$\bigcup_{t \in T} \left(\mathrm{Desc}^*(C) \cap D_{\leq t} \cap V_{\mathrm{exec}}\right).$$

A node may be removed from the actual rebuild set only by certified equality of its complete effective input, an invariant-output theorem, or an admitted cache result.

Proposition 3.9 (C2-v21-DER-04: Multi-target incremental sufficiency)**.** *Under target completeness and hermetic determinism, evaluating the rebuild plan of Definition 3.8 in topological order restores every selected target. Shared stale ancestors are evaluated once and fan out to all target slices.*

Proof. The union contains every structurally affected executable node in each selected target slice, including a changed executable node itself. Topological execution restores each dependency before use, and shared nodes have one immutable output under the hermetic contract. □

Definition 3.10 (C2-v21-DEF-06: Deterministic parallel schedule)**.** A deterministic parallel schedule executes incomparable pure nodes concurrently and commits results through canonical ordering or serializable transactions. It either evaluates every required node or applies a pinned cancellation rule whose cancelled set depends only on the already canonical failure set, not on wall-clock arrival order. The final failure record is a deterministic reduction of the required node outcomes.

Proposition 3.11 (C2-v21-DER-05: Schedule independence)**.** *Every fair topological schedule satisfying Definition 3.10 yields the same canonical target outputs and failure records.*

Proof. Comparable nodes respect dependency order. Incomparable pure nodes do not affect one another, and canonical commit, cancellation, and failure reduction remove dependence on completion timing. □

3.3 Schema migration and semantic replay

Definition 3.12 (C2-v21-DEF-07: Typed schema migration)**.** A typed schema migration is a deterministic adapter with a semantic field map, provenance transformation, supported domain, and consumer ceiling. Default-filled inverses are not lossless unless a theorem proves that the inserted values reconstruct the source semantics.

Proposition 3.13 (C2-v21-DER-06: Lossless round-trip equivalence)**.** *If migrations $M_{a\to b}$ and $M_{b\to a}$ are mutually inverse on declared supported domains and preserve every consumer field and provenance obligation, then deterministic consumers obtain identical results in either schema.*

Proof. The round trip is extensionally identity on the complete effective consumer input. □

Definition 3.14 (C2-v21-DEF-08: Migration confluence certificate)**.** A migration confluence certificate for two paths from schema a to schema d proves equal canonical outputs, equal composite semantic field maps, and equal provenance obligations on their common domain.

Proposition 3.15 (C2-v21-DER-07: Confluent migration invariance)**.** *Under a valid confluence certificate, every deterministic downstream consumer receives the same result along either migration path. Equal endpoint schema names alone are insufficient.*

Proof. The certificate equates the complete effective input and provenance delivered to the consumer. □

Definition 3.16 (C2-v21-DEF-09: Semantic replay morphism)**.** A semantic replay morphism maps a released packet to a replayed packet and preserves the exact target identity, statement, hypotheses, propagated assumption context Γ, scope, source binding, status sort, comparison mode, detector stage, interpretation arrow, artifact role, failure route, supported-strength set $\mathfrak{S}_T$, current-reading map, and every field in the target consumer projection.

Proposition 3.17 (C2-v21-DER-08: Semantic replay composes)**.** *Semantic replay morphisms compose on the domain of the composite semantic field map. Operational byte replay without this morphism does not establish semantic replay.*

Proof. Composition follows from Proposition 1.9. Byte equality alone does not verify the semantic field obligations. □

3.4 Proof-store safety and release closure

Definition 3.18 (C2-v21-DEF-10: Snapshot-safe garbage collection)**.** Snapshot-safe garbage collection fixes an epoch, protected root set, reader pins, root-registration barrier, and deletion barrier. It removes only objects outside the dependency closure of every root protected at physical deletion time.

Proposition 3.19 (C2-v21-DER-09: Live-proof preservation)**.** *Snapshot-safe garbage collection preserves every object required by a protected live proof bundle.*

Proof. Every required object lies in a protected root closure and is excluded from deletion by definition. □

Definition 3.20 (C2-v21-DEF-11: Reproducibility vector)**.** The v21 reproducibility record is the componentwise vector

$$\mathbf{r} = (r_{\text{bytes}}, r_{\text{build}}, r_{\text{compute}}, r_{\text{formal}}, r_{\text{semantic}}, r_{\text{source}}, r_{\text{external}}, r_{\text{audit}}).$$

No scalar summary is allowed to dominate a failed component required by the target.

Definition 3.21 (C2-v21-DEF-14: External-input identity and freshness split)**.** Every external input consumed by a replayable target is frozen by immutable content identity, request semantics, endpoint or model version, retrieval record, and permitted-use policy. Freshness is a separate time-indexed assessment and never substitutes for frozen identity. A later fresh response is a new input and creates a new run identity.

Proposition 3.22 (C2-v21-DER-12: Freshness is not replay identity)**.** *A source or service response described only as current cannot support exact replay. Exact replay consumes the frozen response; a freshness recheck may validate or supersede it but cannot silently replace it inside the old run.*

Proof. Currentness is evaluated at a time and may change. Replay identity requires the same immutable effective input. □

Definition 3.23 (C2-v21-DEF-15: Target-required reproducibility mask)**.** For each target T, the required-component mask M_T names the coordinates of $\mathbf{r}$ that must satisfy their declared acceptance predicates. A scalar R-level, average, minimum-free score, or favorable unused coordinate cannot compensate for failure of a coordinate in M_T.

Proposition 3.24 (C2-v21-DER-13: Componentwise reproducibility acceptance)**.** *A target is reproducible at its declared contract exactly when every coordinate selected by M_T passes its own acceptance predicate. No total ordering of all reproducibility vectors is required.*

Proof. The target contract is the conjunction of its required component predicates. Unused coordinates are outside that conjunction, and failed required coordinates remain failed. □

Definition 3.25 (C2-v21-DEF-12: Reproducibly auditable target closure)**.** A target has reproducibly auditable closure when:

(a) its proof-DAG target slice is certificate-closed;

(b) its proof-store dependency closure is completely resolvable;

(c) its execution envelope, verifier version, canonicalization, and supply chain are hermetic at the declared coordinates;

(d) a semantic replay morphism identifies the released and replayed target consumer projections;

(e) every consumed external input is frozen according to Definition 3.21, while freshness is recorded separately;

(f) every coordinate required by M_T passes;

(g) the independent audit consumes the same target projection and is dependency-separated beyond its declared common base.

Proposition 3.26 (C2-v21-DER-10: Independent reconstruction)**.** *A target satisfying Definition 3.25 can be independently reconstructed to the same canonical target projection and verifier result by an auditor with access to the declared closure. The conclusion is conditional on the pinned deterministic verifier and declared semantic equality; it does not increase mathematical strength or prove that the interpreted theorem is true beyond its source evidence.*

Proof. The closed slice, resolvable store, frozen external inputs, hermetic execution, pinned verifier, semantic replay, and matching audit projection determine the same complete effective consumer input. Determinism gives the same result, while the source theorem fixes its strength ceiling. □

Definition 3.27 (C2-v21-DEF-13: Release handoff packet)**.** A release handoff packet records the frozen document set, content hashes, theorem and local-identity inventories, dependency and ownership matrices, source-binding repairs, migration ancestry, validation reports, unresolved obligations, Claim-Register deferral state, and the exact authority permitted to create the final register.

Proposition 3.28 (C2-v21-DER-11: Release handoff is non-promotional)**.** *A complete release handoff packet does not create canonical Claim UIDs or promote a claim. It makes the frozen inventory, candidate extraction, dependency checks, and unresolved-obligation report reproducible. A final promotion decision is deterministic only when the promotion policy itself is frozen, complete, and mechanical; otherwise the separately authorized governance judgment remains an independent act.*

Proof. The packet records the inputs and admissibility conditions but does not exercise promotion authority or eliminate policy discretion that is not encoded in the packet. □

```
v21_execution_package:
  id: "exec-packet-0001"
  version: "21.0.0"
  calibration_packet: "content-addressed id"
  target_slice: "Merkle-rooted DAG closure"
  hermetic_envelope:
    code: "digest"
    environment: "digest"
    backend: "version-pinned"
    external_inputs:
      frozen: []
      freshness_assessments: []
    randomness: "none / seeded / distributional-contract"
  cache_admission:
    used: false
    certificate: null
  replay:
    operational: "pass / reject / undefined / not_invoked"
    semantic: "pass / reject / undefined / not_invoked"
    source_rebinding: "pass / reject / undefined / not_invoked"
  reproducibility_vector:
    bytes: "declared"
    build: "declared"
    compute: "declared"
    formal: "declared"
    semantic: "declared"
    source: "declared"
    external: "declared"
    audit: "declared"
  release_handoff:
    frozen_documents: []
    unresolved_obligations: []
    claim_register_status: "pending_final_CR"
    package_release_status: "not_released"
    publication_status: "not_published"
  non_claims:
    - "Execution and replay do not manufacture missing mathematics."
    - "Release handoff is not Claim Register promotion."
```

Listing 2: v21 execution and release handoff schema

4 v21 Companion Cross-Closure and Migration

Definition 4.1 (C-v21-DEF-11: Companion closure-state vector)**.** The Companion closure state is the vector

$$\mathbf{g} = (g_{\text{local}}, g_{\text{cross}}, g_{\text{dep}}, g_{\text{handoff}}, g_{\text{CR}}, g_{\text{package}}, g_{\text{publication}}).$$

Local completion precedes crosscheck, dependency freezing, and release-handoff readiness. Claim-Register promotion, package release, and publication are separate downstream axes with their own authorities and may not be collapsed into one total workflow chain. For every affected target, blocked or undefined on a required axis prevents the corresponding downstream action.

Proposition 4.2 (C-v21-DER-14: Release-axis noncollapse)**.** *Claim-Register promotion does not by itself publish or package a release; package release and publication do not by themselves promote a Claim; and release-handoff readiness implies none of the three. Any synchronization among these axes requires an explicit governance rule.*

Proof. The axes have distinct predicates, artifacts, and authorities. No implication is part of their definitions. □

Definition 4.3 (C-v21-DEF-12: Companion migration record)**.** Every Companion migration uses the Main v21 semantic status

inherited, strengthened, repaired, rejected, successor_only,

with compatible disposition modifiers chosen from

merged, relocated, deprecated, source_only.

A separate operational-template disposition records whether the local schema is a retained predecessor, current-reading refinement, replacement schema, superseded template, or v21-only addition. The operational disposition never changes the semantic migration status. The record also names the current owner, supported-strength set, consumer ceiling, predecessor invocability, and Claim-Register state.

Proposition 4.4 (C-v21-DER-10: Conservative v21 migration)**.** *The v21 Companion migration is conservative when every v20 predecessor statement remains available at no stronger than its displayed meaning, every semantic status and operational disposition is separately recorded, every changed current reading is explicit, and every v21 addition has a fresh identity and does not compete with canonical mathematical owners.*

Proof. The predecessor corpus is preserved, while current operational use is governed by separately identified v21 records. No predecessor theorem is silently rewritten or promoted. □

Proposition 4.5 (C-v21-DER-11: Companion closure theorem)**.** *The v21 Companion layer supplies a target-relative operational projection from source-specific calibration through target-sliced execution, proof storage, semantic replay, audit, and release handoff. The projection is lossless on every declared consumer projection and immutable artifact closure, not necessarily on all source fields. It remains noncompensating and nonpromotional: every mathematical conclusion still requires the exact canonical theorem, Bridge component, Return theorem, source authority, and target verifier owned outside the Companion.*

Proof. The preceding definitions provide the operational chain and certify preservation only for explicitly mapped consumer fields. The ownership, supported-strength, and noncompensation rules exclude mathematical promotion or undeclared global losslessness. □

4.1 Explicit v21 exclusions

The v21 Companion asserts none of the following:

- no template, calibration packet, execution packet, route graph, store entry, cache hit, replay, audit, or release handoff is proof by itself;
- no shared schema identifies arithmetic and PDE predicates;
- no route portfolio permits cross-route borrowing from incomplete routes;
- no validation corpus remains independent after adaptive inspection without an adaptive-validity theorem;
- no finite arithmetic samples prove a recurrence or Iwasawa tail before the recurrence theorem is bound;
- no finite local PDE packets prove a global norm or regularity result without the exact aggregation and source criterion;
- no Merkle root, hash, signature, mirror count, or storage authority proves semantic identity or mathematical truth;
- no cache object enters theorem-facing execution without cache admission;
- no schema endpoint name proves migration confluence or lossless round trip;
- no operational replay is semantic replay, and no semantic replay is independent mathematical reconstruction by itself;
- no target is forced to have a unique strongest supported conclusion when its strength preorder has incomparable maxima;
- no handoff token is replaced by P2DG readiness or an atomic status;
- no external input described only as current supplies exact replay identity;
- no positive closure coordinate, including release handoff readiness, creates a canonical Claim UID, package release, or publication event;
- no Claim Register is generated by this Companion layer.

Immutable v20 Companion Predecessor Corpus

The following Companion 1 and Companion 2 corpus is preserved verbatim as predecessor ancestry. Its v20 wording remains historically available; current v21 consumption is governed by the synchronization and reinforcement layer above.

Preservation record.

- Current v21 local items: 78 total, comprising 41 definitions and 37 operational propositions.
- Re-audit disposition relative to the preceding v21 edition: 43 retained, 25 corrected or strengthened, and 10 newly added.
- Frozen predecessor labels: 134/134, all unique.
- Frozen predecessor environments: 66 definitions, 25 propositions, and 41 remarks.
- The predecessor body, beginning with Companion 1 and ending with $\mathrm{end}\{\mathrm{document}\}$, is included exactly once and is byte-for-byte identical to the submitted v20 source body.
- Frozen body SHA-256:

  $\mathrm{d58ed55198b0a8c4daa5bc9f855fd628da97550ec681a6bef9e125c712e2aa73}$

- Current v21 readings, ownership boundaries, and operational schemas are supplied only by the preceding v21 layer; no predecessor label is rebound.

1 Companion 1: Arithmetic Calibration Companion (v20.0.0)

1.1 C1.1. Scope and non-theorem status

This companion document provides operational calibration material for the arithmetic interface appendices of the AK framework. It is a companion document, not a theorem-bearing appendix. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this companion is:

Arithmetic calibration examples are not arithmetic theorems.

The purpose of this companion is to provide templates, calibration records, example schemas, failure patterns, and non-claim examples for arithmetic interfaces. It supports the use of:

Appendix MS-A, Appendix MS-B, Appendix MS-C, Appendix MS-F.

It does not prove the Birch–Swinnerton-Dyer conjecture, the ABC conjecture, the Riemann hypothesis, Iwasawa main conjectures, modularity theorems, congruence theorems, mirror symmetry, or Fukaya-categorical theorems.

No calibration record, example file, arithmetic template, tower record, bridge sketch, or operational manifest may replace:

$$B\text{-Gate}^{+}, \quad \mathrm{Eval}_{n,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}).$$

Remark 1.1 (Companion status)**.** This document is operational and illustrative. It may contain templates and examples that help instantiate the arithmetic interface protocol. It does not create Core theorem status, interface theorem status, bridge theorem status, or arithmetic theorem status.

Remark 1.2 (Relation to Part IV)**.** Part IV defines the problem-interface discipline. This companion supplies calibration material for using that discipline in arithmetic settings. When in doubt, the formal definitions and exclusions in Appendices MS-A–MS-F take precedence over examples in this companion.

Remark 1.3 (Companion boundary)**.** A companion may show how a record should be shaped. It may not promote the record from example status to theorem status. Promotion requires the relevant verifier-side or bridge-theorem evidence.

1.2 C1.1bis. v20 companion hardening and verifier-side boundary

Definition 1.4 (Companion-sovereignty boundary)**.** The companion-sovereignty boundary is the rule that an arithmetic calibration object, example schema, mock record, bridge sketch, template, or non-claim example remains operational companion material unless it is converted into a declared verifier-side or bridge-theorem record and passes the applicable protocol. This boundary applies even when the example is internally well formed, checked by a human, stored in a platform, or copied into an audit manifest.

Definition 1.5 (Calibration-to-verifier conversion record)**.** A calibration-to-verifier conversion record is required whenever a calibration object is consumed as more than an example. It declares the source calibration object, target verifier-side schema, source and target scopes, required hypotheses, artifact identities, claim-use preflight result, resulting status $pass/reject/undefined$, and typed failure route. Without this record, the calibration object remains a companion template or non-claim example.

Definition 1.6 (Companion scope-exclusion policy)**.** A template field may be written as $\mathrm{not\_applicable}$ or $\mathrm{not\_used}$ for compatibility. It maps to $\mathrm{not\_invoked}$ only when a declared scope-exclusion policy explains why the clause is outside the run scope. Absent that policy, theorem-facing consumption is $\mathrm{undefined}$, not pass.

Proposition 1.7 (Checked calibration is non-verdict)**.** *A checked calibration does not determine Core pass, interface theorem status, bridge theorem status, or arithmetic theorem status.*

Proof. Checked calibration verifies companion-level well-formedness only. Core and bridge verdicts consume verifier-side records, artifacts, ledgers, diagnostics, claim-use preflight, and proofs at their declared types. A companion-level shape check supplies none of those obligations by itself. □

Proposition 1.8 (Companion examples cannot discharge non-scalar obligations)**.** *No arithmetic companion example discharges representative existence, eligibility, terminal tameness, artifact identity, claim-use permission, or tower-comparison existence.*

Proof. These are non-scalar verifier-side obligations under the v20 Core guard-rails. An example can illustrate the field in which evidence would be recorded, but it is not the evidence itself. □

Remark 1.9 (Companion ledger-token discipline)**.** Illustrative local ledger tokens include

$$\mathrm{delta\_arith}, \quad \mathrm{delta\_cong}, \quad \mathrm{delta\_Iw}, \quad \mathrm{delta\_tower}, \quad \mathrm{delta\_bridge}.$$

They enter a metric budget only through Appendix M registration, support records, scalarization, and a bottleneck discrepancy bound.

1.3 C1.1ter. v20 claim axes, dependency closure, and final-CR deferral

Definition 1.10 (Companion claim-axis record)**.** A *companion claim-axis record* keeps independent fields for document role, evidence status, invocation role, conformance status, workflow lifecycle, and the exact verifier-side atomic status of any converted record. A calibration lifecycle value such as checked, replayable, or complete is never copied into mathematical pass.

Definition 1.11 (Companion dependency-closure record)**.** A *companion dependency-closure record* identifies the immutable Main, Foundation, CR-v20 baseline, CM-v20, TB-v20, Technical Extension, Problem Interface, Search / Platform, source-binding, recovery-manifest, and execution artifacts actually consumed by one calibration example. It distinguishes canonical baseline claims from frozen local downstream statements and records every unresolved dependency as undefined rather than silently omitting it.

Definition 1.12 (Companion final-CR deferral record)**.** A *companion final-CR deferral record* states that stable local labels and immutable artifacts may be cross-audited before canonical Claim UID assignment, but no local table, generated JSON/CSV record, calibration status, or companion closure statement is represented as final CR-v20 promotion. Canonical promotion is deferred until the Companion and global dependency closure are frozen and the final register is generated under its own authority.

Proposition 1.13 (Companion completion is non-promotional)**.** *Completion, checking, replay, or publication of a companion template does not promote the illustrated mathematical claim or create a canonical Claim UID.*

Proof. A companion record verifies only its declared operational shape and provenance. Mathematical promotion additionally requires the exact theorem evidence, source binding, discharged hypotheses, dependency closure, permitted use, and register authority. □

Remark 1.14 (v20 dependency position)**.** The Main and Foundation Appendices are consumed through the CR-v20 baseline. CM-v20, TB-v20, Technical Extension H–N, Problem Interface Part IV, and Search / Platform Part V are consumed only at their frozen local strengths and immutable artifact identities until the final CR-v20 is produced. The present Companion is the last construction-stage document before the global cross-audit and final register extraction.

1.4 C1.2. Calibration object taxonomy

Definition 1.15 (Arithmetic calibration object)**.** An *arithmetic calibration object* is an illustrative or operational object used to test, configure, or document an arithmetic interface workflow. Examples include:

(i) a sample arithmetic source datum;

(ii) a sample cyclotomic tower;

(iii) a sample Iwasawa module profile;

(iv) a sample congruence defect record;

(v) a sample realization template;

(vi) a sample repaired evaluation record;

(vii) a sample admissible representative record;

(viii) a sample tower comparison artifact record;

(ix) a sample ledger entry;

(x) a sample Type IV proxy record;

(xi) a sample monitored diagnostic record;

(xii) a sample manifest entry;

(xiii) a failure-case record.

Definition 1.16 (Calibration status)**.** A calibration object has one of the following statuses:

(i) example;

(ii) template;

(iii) mock record;

(iv) calibration candidate;

(v) checked calibration;

(vi) failed calibration;

(vii) non-claim example;

(viii) deprecated example;

(ix) superseded example.

Definition 1.17 (Checked calibration)**.** A *checked calibration* is a calibration object whose fields are internally well-formed for the declared companion-level purpose. It is not a Core certificate unless separately submitted to and accepted by the Core verifier.

Remark 1.18 (Calibration is not certification)**.** A calibration object may help test whether an interface record is well-formed. It does not certify the corresponding arithmetic claim.

1.5 C1.3. Arithmetic source data template

An arithmetic interface should begin by declaring its source data.

Definition 1.19 (Arithmetic source data calibration template)**.** An *arithmetic source data calibration template* has the following fields:

(i) source identifier;

(ii) arithmetic family;

(iii) base field or base scheme;

(iv) coefficient field k;

(v) arithmetic objects;

(vi) local conditions, if any;

(vii) tower structure, if any;

(viii) congruence structure, if any;

(ix) realization purpose;

(x) intended Core-readable target, if any;

(xi) non-claim declaration.

A suggested schematic record is:

```
arithmetic_source:
  id: "arith-src-001"
  family: "elliptic_curve / abelian_variety / galois_representation / selmer_complex / iwasawa_module /
      congruence_family"
  base: "declared global field or arithmetic base"
  coefficient_field: "k"
  objects:
    - "object_1"
    - "object_2"
  local_conditions:
    status: "declared / not_used / unavailable"
    references: []
  tower:
    status: "none / cyclotomic / level / congruence / other"
    direction: "finite_to_limit / local_to_global / scale_refinement"
  congruence:
    status: "none / declared / advisory"
  realization_purpose: "PH_n / Ext^n-edge / tower diagnostic / ledger comparison / advisory"
  core_target:
    status: "none / FiltCh(k) / Perscons_k / Eval_{i,W,tau} / C_{tau,W}F / phi_{i,W,tau}"
  non_claim:
    - "This source declaration does not prove an arithmetic theorem."
```

Remark 1.20 (Purpose must be explicit)**.** The same arithmetic object may support different interface purposes. For example, a Selmer-related object may be used for an advisory obstruction profile, a persistence realization, a tower diagnostic candidate, or a bridge candidate. The intended purpose must be declared before calibration can be evaluated.

1.6 C1.3bis. Source binding and Known-Theorem Recovery calibration

Definition 1.21 (Arithmetic source-binding calibration record)**.** An *arithmetic source-binding calibration record* contains the authoritative source identity, exact theorem or precursor locator, immutable local statement-binding artifact, content identity, source role, permitted use, prohibited stronger reading, and freshness status. A model summary, search result, transcription, manifest row, or citation string is not the authoritative source merely because it reproduces the intended words.

Definition 1.22 (Known-Theorem Recovery calibration record)**.** A *Known-Theorem Recovery calibration record* separates:

(i) the immutable benchmark statement and exact locator;

(ii) the permitted precursor package;

(iii) source-to-AK realization and coefficient passage;

(iv) the finite Core certificate;

(v) any finite-to-infinite reduction;

(vi) the separately proved Return theorem;

(vii) final independent exact-strength validation;

(viii) prohibited conclusion-side inputs and the non-circularity audit.

The benchmark statement may specify the target and the consequent of Return, but the benchmark proof, known truth value, target constant, exceptional bound, conclusion-derived witness, and equivalent oracle are unavailable to the construction of the preceding arrows.

Proposition 1.23 (Benchmark quarantine is non-compensating)**.** *A calibration that violates benchmark quarantine cannot be repaired by agreement with the benchmark, replay success, metric reserve, manifest completeness, or human approval.*

Proof. The defect is circularity in the semantic origin of the evidence. The listed favorable conditions concern different predicates and do not reconstruct an independent derivation. □

Remark 1.24 (v20 exact-strength reference tracks)**.** The frozen local tracks IW-KTR-GROWTH and NS-KTR-SERRIN-L6 may be used as exact-strength calibration examples. The first is relative to its registered Iwasawa precursor package and recovers only the characteristic-growth clause; the second recovers only the one-way fixed-exponent strict-subcritical Serrin implication. Neither track proves an Iwasawa main conjecture, class-number specialization, new analytic regularity theorem, unconditional Navier–Stokes regularity, Type IV cleanliness, or general information preservation.

1.7 C1.4. Cyclotomic tower calibration

Definition 1.25 (Cyclotomic tower calibration record)**.** A *cyclotomic tower calibration record* documents how a cyclotomic or p-power tower is represented for Core-readable analysis. It must record:

(i) tower layers;

(ii) transition maps;

(iii) finite-stage arithmetic data;

(iv) limiting object policy;

(v) realization route;

(vi) repaired evaluation policy;

(vii) tower comparison artifact target;

(viii) monitored degrees;

(ix) defect and ledger treatment;

(x) non-claim declarations.

A suggested schematic record is:

```
cyclotomic_tower_calibration:
  id: "cyc-cal-001"
  prime: "p"
  layers:
    indexing: "n >= 0"
    layer_objects: ["A_0", "A_1", "..."]
    transition_maps: "declared"
  limiting_policy:
    object: "A_infty"
    status: "declared / conjectural / not_available"
  realization:
    functor_or_procedure: "R_cyc"
    target: "FiltCh(k) / Perscons_k"
    constructibility_check: "pass / reject / undefined"
  repaired_evaluation:
    expression: "Eval_{i,W,tau}(F_n) = T_bar_tau(W_W P_i(F_n))"
    monitored_degrees: ["declared finite degree set"]
    windows: []
    threshold_policy: "declared"
  tower_artifact:
    comparison_artifact: "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(
        F_infty)"
    artifact_status: "candidate / declared / verified / rejected / undefined"
  ledger:
    delta_cyc: "value_or_bound"
    aggregation: "declared ledger semantics"
  non_claim:
    - "This calibration does not prove a cyclotomic control theorem."
    - "This calibration does not prove Type IV cleanliness."
```

Remark 1.26 (Cyclotomic tower versus Type IV tower)**.** A cyclotomic tower is not automatically a Type IV diagnostic tower. It becomes diagnostically meaningful only after a Core-readable realization, repaired evaluation policy, and declared tower comparison artifact

$$\phi_{i,W,\tau}$$

are supplied.

1.8 C1.5. Iwasawa module calibration

Definition 1.27 (Iwasawa module calibration record)**.** An *Iwasawa module calibration record* documents how an Iwasawa module or Iwasawa-layer object is represented in the AK interface. It must record:

(i) Iwasawa algebra or tower;

(ii) module or complex;

(iii) arithmetic μ-invariant convention, if used;

(iv) one-parameter filtration extraction;

(v) Core-readable target;

(vi) repaired evaluation policy;

(vii) distinction from μ_{Collapse};

(viii) transport status.

A suggested schematic record is:

```
iwasawa_calibration:
  id: "iw-cal-001"
  iwasawa_algebra: "Lambda"
  module: "M_Iw"
  arithmetic_mu:
    symbol: "mu_Iw"
    convention: "declared arithmetic convention"
    value_or_status: "known / unknown / conjectural / not_used"
  filtration:
    parameter: "cyclotomic_height / p_power_level / conductor_depth / congruence_depth"
    extraction_rule: "declared"
  realization:
    target: "FiltCh(k) / Perscons_k"
    core_readable: "yes / no / pending"
  repaired_evaluation:
    expression: "Eval_{i,W,tau}(F_Iw)"
    status: "not_invoked / computed / undefined / rejected"
  collapse_mu:
    symbol: "mu_Collapse"
    diagnostic_source: "kernel defect profile of phi_{i,W,tau}"
    status: "not_computed / computed / pending"
  transport:
    mu_Iw_to_mu_Collapse: "[Spec] / proved / rejected / not_claimed"
    mu_Collapse_to_mu_Iw: "[Spec] / proved / rejected / not_claimed"
  non_claim:
    - "mu_Iw is not identified with mu_Collapse by default."
    - "This calibration does not prove an Iwasawa main conjecture."
```

Remark 1.28 (Mandatory symbol separation)**.** The calibration record should never write simply $\mu = 0$ when both μ_{Iw} and μ_{Collapse} are in scope. The subscript is mandatory.

1.9 C1.6. Congruence defect calibration

Definition 1.29 (Congruence defect calibration record)**.** A *congruence defect calibration record* documents how congruence data are compared after realization and how any discrepancy is ledgered. It must record:

(i) congruence source data;

(ii) congruence relation;

(iii) realization route;

(iv) compared realized objects;

(v) repaired evaluation comparison, if used;

(vi) congruence defect;

(vii) ledger entry;

(viii) Type IV proxy status, if any;

(ix) non-claim declarations.

A suggested schematic record is:

```
congruence_defect_calibration:
  id: "cong-cal-001"
  source_objects:
    - "A"
    - "A_prime"
  congruence_relation: "A congruent to A_prime modulo q / ideal / level"
  realization:
    R_cong_A: "F_A"
    R_cong_A_prime: "F_A_prime"
    target: "Perscons_k / FiltCh(k)"
  repaired_evaluation_comparison:
    left: "Eval_{i,W,tau}(F_A)"
    right: "Eval_{i,W,tau}(F_A_prime)"
    relation: "equal / interleaved / bounded_defect / failed / unknown"
  comparison:
    defect_symbol: "delta_cong"
    defect_value_or_bound: "declared"
  ledger:
    include_in_metric_ledger: true
    aggregation: "declared"
  typeIV_proxy:
    status: "none / advisory / candidate / computed_diagnostic"
    warning: "congruence failure is not Type IV diagnosis"
  non_claim:
    - "Congruence failure is not identified with Type IV obstruction by default."
    - "This calibration does not prove a congruence theorem."
```

Remark 1.30 (Congruence is not persistence equality)**.** Arithmetic congruence does not automatically imply equality or interleaving of realized persistence objects. The realized comparison must be separately declared and supported.

1.10 C1.7. Arithmetic realization template

Definition 1.31 (Arithmetic realization calibration template)**.** An *arithmetic realization calibration template* records the route by which arithmetic data become Core-readable. It must include:

(i) source data;

(ii) realization procedure;

(iii) target category;

(iv) filtration parameter;

(v) constructibility check;

(vi) window policy;

(vii) threshold policy;

(viii) monitored degrees;

(ix) repaired evaluation policy;

(x) purpose-faithfulness statement;

(xi) artifact references.

A suggested schematic record is:

```
arithmetic_realization:
  id: "arith-real-001"
  source: "arith-src-001"
  procedure: "R_arith"
  target_category: "FiltCh(k) / Perscons_k / D^{[-n,0]}(k-mod)"
  filtration:
    parameter: "declared"
    direction: "increasing / decreasing"
    normalization: "declared / none"
  constructibility:
    status: "pass / reject / undefined"
    evidence: []
  windows:
    declared_windows: []
    MECE_status: "pass / reject / undefined"
  threshold_policy:
    tau: "declared"
    endpoint_policy: "declared"
  monitored_degrees: []
  repaired_evaluation:
    expression: "Eval_{i,W,tau}(F)"
    status: "not_applicable / candidate / computed / verified / failed"
  purpose_faithfulness:
    target_feature: "Selmer obstruction / congruence defect / tower growth / advisory"
    status: "[Spec] / proved / operational"
  artifacts: []
  non_claim:
    - "This realization does not prove the arithmetic target theorem."
```

Remark 1.32 (Realization can fail)**.** A realization may fail because it is not constructible, not one-parameter, not purpose-faithful, not eligible, not compatible with repaired evaluation, or not artifact-supported. Failure should be recorded rather than hidden.

1.11 C1.8. Eligible regime calibration

Definition 1.33 (Eligible regime calibration record)**.** An *eligible regime calibration record* documents whether a realized arithmetic object satisfies the requirements for invoking the standard degree-n Ext^n-edge. It must record:

(i) admissible representative;

(ii) compatibility with repaired evaluation;

(iii) realization into $D^{\mathrm{b}}(k\text{-mod})$;

(iv) amplitude check;

(v) edge-realization check;

(vi) residual PH_1 object;

(vii) Ext^n-clause status.

A suggested schematic record is:

```
eligible_regime_calibration:
  id: "elig-cal-001"
  input_object: "F"
  window: "W"
  threshold: "tau"
```

```
admissible_representative:
  object: "C_{tau,W}F"
  status: "declared / constructed / failed"
compatibility:
  required: "P_i(C_{tau,W}F) ~= Eval_{i,W,tau}(F)"
  status: "pass / reject / undefined"
realization:
  object: "R(C_{tau,W}F)"
  ambient_category: "D^b(k-mod)"
amplitude:
  required: "[-n,0]"
  status: "pass / reject / undefined"
edge_realization:
  required: "H^{-n}(R(C_{tau,W}F)) ~= E_n(R(C_{tau,W}F))"
  status: "pass / reject / undefined"
ext_clause:
  allowed: "yes / no"
  reason: "declared"
non_claim:
  - "Arithmetic Ext is not automatically Core Ext."
  - "Ext => PH is not used as a Core theorem."
```

Remark 1.34 (No eligibility, no Ext^n-edge)**.** If admissible representative compatibility, amplitude, or edge realization fails, the standard degree-n Ext^n-edge cannot be invoked. The correct status is failure or pending, not informal acceptance.

1.12 C1.8bis. Standard degree-n eligibility and edge calibration

Definition 1.35 (Standard degree-n edge calibration record)**.** For one invoked degree n, a *standard degree-n edge calibration record* contains the exact repaired degree-n profile, admissible representative, actual profile isomorphism at the stated strength, genuine realization or separately proved admissible replacement, realization-domain and amplitude checks, a fixed extractor E_n with $E_n(0) = 0$, an artifact-backed identification of H^{-n} with the extractor, naturality and change-compatibility scope, and immutable proof artifacts. Its default theorem direction is

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0.$$

Proposition 1.36 (Reverse higher-edge use requires reflection)**.** *A calibration may not infer repaired-profile vanishing from* Ext^n*-vanishing unless a separately proved natural zero-reflection or faithfulness theorem is bound to the same realization, extractor, and source class.*

Proof. The standard v20 edge is one-way. The converse is a distinct theorem and cannot be supplied by notation, a synthetic shift, or calibration intent. □

Proposition 1.37 (Synthetic realization is not a natural bridge)**.** *A shifted or target-shaped derived object may calibrate a synthetic or toy identity, but it does not establish a purpose-faithful natural* PH_n*-to-*Ext^n *bridge.*

Proof. A synthetic identity need not preserve external source semantics, naturality, representative independence, or the intended Return predicate. Those arrows require independent theorem evidence. □

1.13 C1.9. Arithmetic ledger template

Definition 1.38 (Arithmetic ledger calibration record)**.** An *arithmetic ledger calibration record* documents how arithmetic transfer, comparison, congruence, normalization, discretization, tower, or bridge defects enter the δ-ledger.

A suggested schematic record is:

```
arithmetic_ledger:
  id: "arith-ledger-001"
  ledger_semantics:
    type: "additive / max / weighted / product / hybrid"
    strict_order: "declared"
  components:
    delta_arith:
      source: "arithmetic transfer"
      value_or_bound: "declared"
      artifact: []
    delta_cong:
      source: "congruence defect"
      value_or_bound: "declared"
      artifact: []
    delta_Iw:
      source: "Iwasawa-layer transfer"
      value_or_bound: "declared"
      artifact: []
    delta_tower:
      source: "tower comparison or limiting transfer"
      value_or_bound: "declared"
      artifact: []
    delta_bridge:
      source: "arithmetic bridge defect"
      value_or_bound: "declared"
      artifact: []
  aggregation:
    d_met: "declared metric-ledger aggregate"
    val_B_d_met: "computed_or_declared"
    protected_family: "B_family"
    gap_tau: "declared"
    comparison: "val_B(d_met) < gap_tau(B_family)"
    budget_verdict: "pass / reject / undefined / not_invoked"
  non_claim:
    - "Ledger pass does not prove arithmetic theorem."
    - "Ledger pass does not replace B-Gate+ or diagnostics."
```

Remark 1.39 (Ledger pass is not arithmetic pass)**.** A ledger pass means only that the declared metric-ledger comparison has passed on its declared scope. It does not imply a Selmer statement, Iwasawa statement, congruence theorem, or arithmetic conclusion. Nor does it repair non-scalar obligations such as realization faithfulness, eligibility, terminal tameness, artifact identity, or claim-use permission.

1.14 C1.9bis. Quantale scalarization and selected-sub-DAG ledger discipline

Definition 1.40 (Selected calibration route)**.** A *selected calibration route* P is a finite admissible target sub-DAG containing every branch required by the calibrated proof or bridge chain. It need not be a single linear path. Unused alternative routes are excluded, while every required branch is included.

Definition 1.41 (v20 quantale ledger calibration)**.** Let $(\mathcal{V}, \oplus, 0_{\mathcal{V}}, \preceq)$ be the declared commutative quantale compatible with Appendix M. Let

$$\mathrm{val}_B : \mathcal{V} \longrightarrow [0, +\infty]$$

be monotone and subadditive, with $\mathrm{val}_B(0_{\mathcal{V}}) = 0$. For the finite selected route P, only the finite ordered commutative-monoid reduct is used. After resolving every local name as an alias, refinement, aggregate, or disjoint contribution, set

$$\mathfrak{d}_{C1}(P) = \bigoplus_{e \in S_P} \delta_e.$$

Each primitive metric component binds actual windowed pre-threshold profiles and proves

$$d_B\left(B_e^{\mathrm{src}}, B_e^{\mathrm{tgt}}\right) \leq \mathrm{val}_B(\delta_e).$$

Any other value system enters a Core metric budget only through a certified unit-preserving conversion into the declared Core quantale and a target-side profile-bound support record.

Proposition 1.42 (No companion double counting)**.** *A primitive discrepancy is counted at most once on the selected route, regardless of how many arithmetic, realization, bridge, search, storage, execution, or replay names refer to it.*

Proof. Renaming, mirroring, storing, or replaying one discrepancy does not create a new independent profile comparison. The relation record controls aggregation. □

Proposition 1.43 (Metric reserve does not repair non-scalar obligations)**.** *No scalarized ledger reserve repairs missing source authority, realization, detector completeness, representative existence, higher-edge eligibility, actual tower morphism, terminal tameness, finite-to-infinite reduction, Return, claim permission, artifact identity, or semantic replay.*

Proof. The listed conditions are typed theorem, existence, authority, or governance obligations rather than bottleneck discrepancies. □

1.15 C1.10. Type IV proxy versus monitored diagnostic

Definition 1.44 (Arithmetic Type IV proxy calibration)**.** An *Arithmetic Type IV proxy calibration* records arithmetic signals that may suggest possible monitored Type IV obstruction. It must distinguish proxy status from diagnostic status.

A suggested schematic record is:

```
arithmetic_typeIV_proxy:
  id: "typeIV-proxy-001"
  source:
    type: "Iwasawa growth / congruence anomaly / tower control failure / limiting mismatch"
    description: "declared"
  proxy_status: "advisory / candidate / rejected"
  diagnostic_required:
    tower_artifact: "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(F_infty)"
    kernel_defect: "required"
    cokernel_defect: "required"
    terminal_generic_defect_dimensions: "required"
  monitored_diagnostic:
    mu_Collapse: "not_invoked / value / undefined"
    nu_Collapse: "not_invoked / value / undefined"
    verdict: "certified_zero / obstructed / undefined / not_invoked"
  non_claim:
    - "This proxy is not Type IV diagnosis."
    - "Arithmetic anomaly does not imply mu_Collapse or nu_Collapse nonzero."
```

Remark 1.45 (Proxy discipline)**.** A proxy can tell us where to look. It cannot tell us what the monitored diagnostic is. Only the declared artifact

$$\phi_{i,W,\tau}$$

and its kernel/cokernel defect profiles determine the monitored Type IV diagnostic. The diagnostic record must use the full typed Type IV schema, including status $\mathrm{not\_invoked}$, $\mathrm{undefined}$, $\mathrm{certified\_zero}$, or $\mathrm{obstructed}$; missing data are not encoded as $(\mu_{\mathrm{Collapse}}, \nu_{\mathrm{Collapse}}) = (0, 0)$.

1.16 C1.10bis. Actual-morphism Type IV and transient-defect calibration

Definition 1.46 (v20 Type IV calibration record)**.** A *v20 Type IV calibration record* fixes one (W, τ), a finite monitored degree set I_{mon}, and an actual persistence morphism

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty).$$

It binds source and target semantic authority, transition and colimit data, apex construction, apex-agreement status, terminal-tameness evidence, and full kernel and cokernel profiles. On a valid terminal scope it records

$$\mu_{i,W,\tau} = \mathrm{tgdim}_W(\ker \phi_{i,W,\tau}), \qquad u_{i,W,\tau} = \mathrm{tgdim}_W(\mathrm{coker}\, \phi_{i,W,\tau}),$$

and aggregates only over the same fixed (W, τ):

$$\mu_{\mathrm{Collapse}} = \sum_{i \in I_{\mathrm{mon}}} \mu_{i,W,\tau}, \qquad u_{\mathrm{Collapse}} = \sum_{i \in I_{\mathrm{mon}}} u_{i,W,\tau}.$$

Its specialized status is exactly one of not_invoked, undefined, certified_zero, or obstructed; no numerical pair is assigned in the first two cases.

Definition 1.47 (Transient-defect calibration record)**.** A *transient-defect calibration record* is a separate complete detector record on the full kernel_defect or cokernel_defect profile. It keeps terminal Type IV, full-interior defect, long-transient defect, full profile vanishing, comparison-morphism isomorphism, and apex agreement as distinct predicates.

Proposition 1.48 (Terminal zero does not imply full preservation)**.** *The terminal verdict*

$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$$

does not by itself establish full kernel/cokernel-profile vanishing or transient absence. It also does not establish comparison-morphism isomorphism, apex agreement, or arithmetic information preservation.

Proof. Terminal-generic dimension retains only terminal information on the declared monitored scope. Each stronger conclusion has a different source object or requires an additional theorem. □

1.17 C1.11. Arithmetic bridge calibration

Definition 1.49 (Arithmetic bridge calibration record)**.** An *arithmetic bridge calibration record* documents a proposed or proved bridge between arithmetic meaning and Core-visible behavior. It must record the bridge direction, status, hypotheses, proof obligations, defects, and Core-facing payload if any.

A suggested schematic record is:

```
arithmetic_bridge:
  id: "arith-bridge-001"
  direction: "arithmetic_to_core / core_to_arithmetic"
  statement: "declared bridge statement"
  source_layer: "arithmetic"
  target_layer: "Core / interface / diagnostic"
  status: "[Spec] / bridge_candidate / partially_proved / bridge_theorem / rejected"
  core_payload:
    repaired_evaluation: "Eval_{i,W,tau}(F) / none / pending"
    representative: "C_{tau,W}F / none / pending"
    tower_artifact: "phi_{i,W,tau} / none / pending"
    diagnostics: "mu_Collapse, nu_Collapse / none / pending"
    ledger: "val_B_d_met < gap_tau / none / pending"
  hypotheses:
    - id: "H1"
      statement: "declared"
```

```
    status: "proved / imported / [Spec] / unresolved"
required_lemmas:
  - id: "L1"
    statement: "declared"
    status: "proved / pending / failed"
defects:
  delta_bridge: "value_or_bound / none / pending"
artifacts: []
non_claim:
  - "Bridge Program is not bridge theorem."
  - "Core pass does not imply arithmetic theorem unless this bridge is proved."
```

Remark 1.50 (Direction matters)**.** The directions

$$\text{arithmetic} \Rightarrow \text{Core}$$

and

$$\text{Core} \Rightarrow \text{arithmetic}$$

are distinct. A calibration record must not merge them.

1.18 C1.11bis. Complete Problem-Bridge and Return calibration

Definition 1.51 (Complete arithmetic Problem-Bridge calibration)**.** A *complete arithmetic Problem-Bridge calibration* records independently every invoked arrow in

$$\begin{aligned}\text{authoritative external source} &\longrightarrow \text{purpose-faithful realization} \longrightarrow \text{Core-readable profile}\\ &\longrightarrow \text{finite Core certificate} \longrightarrow \text{finite-to-infinite reduction}\\ &\longrightarrow \text{higher obstruction edge} \longrightarrow \text{Return theorem} \longrightarrow \text{external conclusion.}\end{aligned}$$

An arrow outside immutable run scope is not_invoked; an invoked but unsupported arrow is undefined. Realization and Return are never merged, and a finite witness is not an infinite conclusion without its reduction theorem.

Proposition 1.52 (Every invoked bridge arrow is required)**.** *No calibration map, bridge program, graph path, proof-store bundle, replay result, or manifest licenses the external conclusion when one invoked arrow of the complete chain is missing, reversed, weaker, circular, or scope-mismatched.*

Proof. The conclusion is licensed only by composition of proved arrows with matching source and target semantics. Infrastructure may organize those arrows but cannot create a missing implication. □

1.19 C1.12. Mirror and categorical arithmetic calibration

Definition 1.53 (Mirror/categorical calibration record)**.** A *Mirror/categorical calibration record* documents how an arithmetic or motivic object is compared with a mirror-side or categorical object. It must record comparison cells, realization targets, repaired evaluation targets, and ledger costs.

A suggested schematic record is:

```
mirror_categorical_calibration:
  id: "mir-cal-001"
  source_domain: "arithmetic / motivic / geometric"
  mirror_domain: "categorical / derived / sheaf / Fukaya-side"
  source_object: "A"
  mirror_candidate: "A_dual"
  comparison:
    operation: "Mir / Comp"
    status: "strict / zero_cost / controlled / advisory / unavailable / failed"
```

```
    cells:
      - id: "alpha_Mir"
        type: "2-cell / higher-cell / correspondence"
        cost: "delta(alpha_Mir)"
  realization:
    source_target: "Perscons_k / FiltCh(k) / technical extension"
    mirror_target: "Perscons_k / FiltCh(k) / technical extension"
    core_testable: "yes / no / pending"
  repaired_evaluation:
    source_eval: "Eval_{i,W,tau}(F_source) / none / pending"
    mirror_eval: "Eval_{i,W,tau}(F_mirror) / none / pending"
    comparison_status: "equal / controlled / failed / pending"
  ledger:
    delta_MT: "value_or_bound"
    included: true
  non_claim:
    - "Mirror comparison is not mirror symmetry theorem."
    - "Categorical comparison is not categorical equivalence theorem."
```

Remark 1.54 (Advisory comparison)**.** An advisory mirror comparison may guide search. It may not support Core certification unless converted into strict, zero-cost, or controlled ledgered comparison.

1.20 C1.13. Fukaya-side calibration

Definition 1.55 (Fukaya calibration record)**.** A *Fukaya calibration record* documents how a Fukaya-side structure is treated in an arithmetic or mirror-related interface. It is optional and always non-theorem unless supported by external theorems and a declared bridge.

A suggested schematic record is:

```
fukaya_calibration:
  id: "fuk-cal-001"
  fukaya_source:
    symplectic_object: "X"
    category: "Fuk(X)"
    auxiliary_data:
      coefficients: "declared"
      grading: "declared"
      brane_data: "declared"
      perturbation_data: "declared"
  extraction:
    route: "action_filtration / energy_filtration / microlocal_sheaf / other"
    target: "FiltCh(k) / Perscons_k"
    constructible: "pass / reject / undefined"
  repaired_evaluation:
    expression: "Eval_{i,W,tau}(F_Fuk)"
    status: "candidate / computed / verified / failed / pending"
  comparison:
    mirror_side: "declared"
    comparison_status: "strict / zero_cost / controlled / advisory / failed"
    delta_Fuk_Mir: "value_or_bound"
  non_claim:
    - "This calibration does not construct a Fukaya category."
    - "This calibration does not prove homological mirror symmetry."
    - "Fukaya obstruction is not Core obstruction by default."
```

1.21 C1.14. Failure-case catalog

Definition 1.56 (Arithmetic calibration failure)**.** An *arithmetic calibration failure* occurs when a calibration record cannot support the intended interface use.

Common failure cases include:

(F1) *Nonconstructible realization.* The arithmetic realization does not produce an object in $\mathsf{Pers}_k^{\mathrm{cons}}$.

(F2) *Missing one-parameter filtration.* The source data have several parameters, but no declared one-parameter extraction.

(F3) *Unproved purpose-faithfulness.* The realization is intuitive but not shown to preserve the relevant arithmetic feature.

(F4) *Repaired evaluation failure.* The record claims a value of $\mathrm{Eval}_{i,W,\tau}(F)$, but the supporting evaluation data are absent or inconsistent.

(F5) *Eligibility failure.* The realized object is not in $D^{[-n,0]}(k\text{-mod})$, or the edge-realization condition fails.

(F6) *μ-collision.* μ_{Iw} and μ_{Collapse} are conflated.

(F7) *Proxy inflation.* A congruence anomaly or Iwasawa growth signal is treated as monitored Type IV diagnosis.

(F8) *Tower artifact failure.* A Type IV diagnostic is invoked without a declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

(F9) *Unledgered transfer defect.* A realization or comparison introduces discrepancy but no ledger component.

(F10) *Bridge inflation.* A Bridge Program or bridge candidate is treated as a bridge theorem.

(F11) *Artifact insufficiency.* The record references artifacts that do not reconstruct the claim.

(F12) *Arithmetic theorem inflation.* A Core pass is described as an arithmetic theorem without a proved Core-to-arithmetic bridge.

Remark 1.57 (Failure is useful)**.** Calibration failure is not merely negative. It identifies which part of the interface requires repair.

1.22 C1.15. Non-claim examples

The following are recommended non-claim clauses for arithmetic calibration records.

```
non_claims:
  - "This record is a calibration example, not a theorem."
  - "This record does not prove BSD, ABC, RH, or any Iwasawa main conjecture."
  - "Arithmetic realization is not assumed faithful unless separately proved."
  - "Arithmetic Ext is not automatically Core Ext."
  - "mu_Iw is not mu_Collapse by default."
  - "Congruence failure is not Type IV diagnosis by default."
  - "Core pass does not imply arithmetic conclusion without a proved transport theorem."
  - "Search artifacts do not update Core verdicts."
  - "AI confidence is not proof."
  - "Ledger pass does not replace B-Gate+."
  - "Ledger pass does not replace Eval_{n,W,tau}(F)=0."
  - "Ledger pass does not replace monitored Type IV diagnostics."
  - "A Type IV proxy does not replace phi_{i,W,tau}."
```

Remark 1.58 (Non-claims should be copied into examples)**.** Calibration examples should explicitly include non-claim fields. This reduces the risk that operational examples are later misread as theorem claims.

1.23 C1.16. Minimal arithmetic calibration manifest

Definition 1.59 (Minimal arithmetic calibration manifest)**.** A *Minimal Arithmetic Calibration Manifest* is a structured record containing the minimum fields needed to audit an arithmetic calibration example. It contains:

(i) calibration identifier;

(ii) source data record;

(iii) realization record;

(iv) repaired evaluation record, if used;

(v) admissible representative record, if used;

(vi) window and threshold policy;

(vii) constructibility status;

(viii) eligibility status, if a degree-n Ext^n clause is invoked;

(ix) ledger record;

(x) Type IV proxy or diagnostic record;

(xi) tower comparison artifact record, if diagnostics are invoked;

(xii) bridge status, if any;

(xiii) artifact references;

(xiv) non-claim list;

(xv) status label.

A compact schematic manifest is:

```
arithmetic_calibration_manifest:
  id: "arith-cal-manifest-001"
  status: "example / template / checked_calibration / failed_calibration"
  source: "arith-src-001"
  realization: "arith-real-001"
  repaired_evaluation:
    used: true
    record: "eval-record-001"
    expression: "Eval_{i,W,tau}(F)"
    status: "candidate / computed / verified / failed / pending"
  representative:
    used: false
    record: "C_{tau,W}F / none"
    status: "not_applicable / pass / fail / pending"
  windows: []
  threshold_policy: "declared"
  constructibility: "pass / reject / undefined"
  eligibility:
    used: false
    status: "not_applicable / pass / fail / pending"
  ledger: "arith-ledger-001"
  typeIV:
    proxy_record: "typeIV-proxy-001"
    tower_artifact: "phi_{i,W,tau} / none / linked / pending"
    monitored_diagnostic: "not_computed / linked / pending"
```

```
bridge:
  status: "none / [Spec] / candidate / theorem / rejected"
  record: "arith-bridge-001"
artifacts: []
non_claims: []
```

Remark 1.60 (Calibration manifest versus Minimal Manifest)**.** The Minimal Arithmetic Calibration Manifest is not the same as the Core Minimal Manifest of Appendix G. It is a companion-level operational manifest for examples and calibration records. A Core certificate still requires Appendix G.

1.24 C1.17. Recommended calibration workflow

A recommended arithmetic calibration workflow is:

(C1) Declare arithmetic source data.

(C2) Declare the intended interface purpose.

(C3) Choose a realization route.

(C4) Check one-parameter constructibility.

(C5) Declare windows and thresholds.

(C6) If Core-readable evaluation is used, record

$$\mathrm{Eval}_{i,W,\tau}(F).$$

(C7) If representatives are invoked, record

$$C_{\tau,W}F.$$

(C8) If Ext^n is used, check eligibility.

(C9) Record arithmetic transfer or comparison defects.

(C10) Build the arithmetic ledger.

(C11) Distinguish proxies from monitored diagnostics.

(C12) If monitored Type IV diagnostics are invoked, declare

$$\phi_{i,W,\tau}.$$

(C13) Declare bridge candidates and statuses.

(C14) Add non-claim clauses.

(C15) Attach artifacts.

(C16) Assign calibration status.

(C17) If moving toward certification, create a separate Core certificate record under Appendix G.

Remark 1.61 (Workflow boundary)**.** The calibration workflow ends before Core certification. Moving from calibration to certification requires a separate certificate record and verifier-side checks.

1.25 C1.17bis. Companion handoff to execution and global closure

Definition 1.62 (Calibration-to-execution handoff record)**.** A *calibration-to-execution handoff record* binds each Companion 1 template to the Companion 2 execution schema that can instantiate it. It records immutable template identity, exact source and target schemas, status-axis preservation, semantic-diff requirements, detector stage, comparison mode, selected target sub-DAG, source and benchmark quarantine, required proof artifacts, and prohibited theorem promotion.

Proposition 1.63 (Handoff preserves non-claim status)**.** *Serializing or executing a calibration template through Companion 2 does not promote it beyond its Companion 1 role.*

Proof. Execution changes representation and operational state, not the mathematical evidence status of the illustrated claim. Promotion requires a separate verifier-side conversion and claim-use preflight. □

1.26 C1.18. Summary of Arithmetic Calibration Companion

Proposition 1.64 (Companion 1 calibration package)**.** *As an operational companion document, this companion provides the following calibration package.*

(i) Arithmetic calibration objects are examples, templates, or operational records, not theorems.

(ii) Arithmetic source data should be declared with explicit purpose and non-claim fields.

(iii) Cyclotomic tower calibration must distinguish arithmetic towers from Type IV diagnostic towers.

(iv) Iwasawa calibration must distinguish μ_{Iw} *from* μ_{Collapse}.

(v) Congruence calibration must distinguish congruence defect from monitored Type IV diagnosis.

(vi) Arithmetic realization calibration must declare target category, filtration, constructibility, repaired evaluation, and purpose-faithfulness.

(vii) Eligible regime calibration is required before invoking the standard degree-n Ext^n*-edge.*

(viii) Arithmetic ledger calibration must record defects, aggregation semantics, gap comparison, and non-claims.

(ix) Type IV proxy calibration must distinguish proxy status from monitored diagnostic status.

(x) Monitored Type IV diagnostic calibration requires the tower comparison artifact

$$\phi_{i,W,\tau}.$$

(xi) Bridge calibration must distinguish bridge candidates, Bridge Programs, and bridge theorems.

(xii) Mirror, categorical, and Fukaya-side calibration records are non-theorem interface support records.

(xiii) Failure-case records should classify failures explicitly.

(xiv) Non-claim clauses should be included in calibration examples.

(xv) A Minimal Arithmetic Calibration Manifest supports audit of calibration examples but does not replace the Core Minimal Manifest.

(xvi) Known-Theorem Recovery calibration separates the target statement, permitted precursor package, independent construction, Return, and final validation.

(xvii) Standard degree-n edge calibration is one-way by default and requires the complete eligibility package.

(xviii) Type IV terminal, transient, full-defect, isomorphism, and apex-agreement predicates remain separate.

(xix) Metric calibration uses a selected finite target sub-DAG, actual profile-discrepancy bounds, quantale scalarization, and no-double-counting relations.

(xx) Companion completion and execution handoff do not create final CR-v20 promotion.

Proof. This theorem summarizes operational definitions and templates provided in this companion. Since the companion is non-theorem-bearing with respect to the AK Core, the proof is a structural verification of the companion's calibration roles rather than a mathematical theorem about arithmetic objects. □

1.27 C1.19. Explicit exclusions

The following are not asserted in Companion 1.

- No arithmetic theorem is proved.
- No Birch–Swinnerton-Dyer statement is proved.
- No ABC statement is proved.
- No Riemann hypothesis statement is proved.
- No Iwasawa main conjecture is proved.
- No cyclotomic control theorem is proved.
- No congruence theorem is proved.
- No modularity theorem is proved.
- No mirror symmetry theorem is proved.
- No Fukaya-categorical theorem is proved.
- No arithmetic realization is assumed faithful by example.
- No calibration record is a Core certificate.
- No calibration record replaces
$$\mathrm{Eval}_{n,W,\tau}(F) = 0.$$
- No calibration record replaces admissible representative records
$$C_{\tau,W}F.$$
- No calibration ledger pass is an arithmetic theorem.
- No arithmetic Type IV proxy is monitored Type IV diagnosis.
- No Type IV diagnostic is accepted without a declared tower comparison artifact
$$\phi_{i,W,\tau}.$$
- No equality
$$\mu_{\mathrm{Iw}} = \mu_{\mathrm{Collapse}}$$
is asserted by calibration.

- No Core pass is interpreted as arithmetic theorem without a proved transport theorem.
- No Companion document enlarges the Core theorem package.
- No template field marked $\mathrm{not\_applicable}$ or $\mathrm{not\_used}$ becomes $\mathrm{not\_invoked}$ without a declared scope-exclusion policy.
- No local companion defect token enters the Core metric budget without Appendix M catalog registration, support record, scalarization, and bottleneck discrepancy evidence.
- No finite observation, finite prefix, plateau, or stored apex proves finite-to-infinite reduction or apex agreement.
- No terminal Type IV zero proves transient absence, full defect vanishing, morphism isomorphism, or arithmetic information preservation.
- No benchmark proof, known answer, conclusion-derived witness, or equivalent oracle enters a Known-Theorem Recovery construction.
- No synthetic degree shift is promoted to a natural higher-n bridge.
- No companion closure grade is represented as final CR-v20 or release promotion.

1.28 C1.20. Provenance and status

This companion depends on:

(i) Appendix MS-A: Arithmetic Interface Meta-Theory;

(ii) Appendix MS-B: Cyclotomic / Iwasawa / Congruence Interfaces;

(iii) Appendix MS-C: Mirror Reflection and Categorical Comparison;

(iv) Appendix MS-F: Fukaya-Categorical Extension;

(v) Appendix A: Repaired Barcode-Level Collapse and Constructible Persistence;

(vi) Appendix B: Admissible Representatives;

(vii) Appendix C: Eligible Realization Regime;

(viii) Appendix D: Tower Diagnostics and Type IV Defect Calculus;

(ix) Appendix G: Minimal Manifest and Proof Object Schema;

(x) Appendix M: Ledger Semantics and Integrated Defect Catalog;

(xi) Appendix U: Agent Semantics;

(xii) Appendix W: Bridge Programs;

(xiii) Appendix Z: Execution and Reproducibility Schema.

Its role is to support disciplined arithmetic interface operation without converting examples, templates, or calibration records into theorem claims.

Status labels for Companion 1.

Operational companion definitions: Definitions 1.15, 1.16, 1.17, 1.19, 1.25, 1.27, 1.29, 1.31, 1.33, 1.38, 1.44, 1.49, 1.53, 1.55, 1.56, 1.59, 1.4, 1.5, 1.6.
Operational companion propositions: Propositions 1.7, 1.8, and 1.64.
Operational cautions: Remarks 1.1, 1.2, 1.3, 1.18, 1.20, 1.26, 1.28, 1.30, 1.32, 1.34, 1.39, 1.45, 1.50, 1.54, 1.57, 1.58, 1.60, 1.61, 1.9.
v20 strengthened companion definitions: Definitions 1.10, 1.11, 1.12, 1.21, 1.22, 1.35, 1.40, 1.41, 1.46, 1.47, 1.51, 1.62.
v20 strengthened companion propositions: Propositions 1.13, 1.23, 1.36, 1.37, 1.42, 1.43, 1.48, 1.52, 1.63.
v20 strengthened companion remarks: Remarks 1.14, 1.24.

Explicit exclusions: Section C1.19.

2 Companion 2: Execution / Reproducibility Companion (v20.0.0)

2.1 C2.1. Scope and non-proof status

This companion document provides operational templates and examples for execution, reproducibility, replay, proof-store records, and audit workflows in the AK framework. It is a companion document, not a theorem-bearing appendix. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this companion is:

Execution and reproducibility templates are operational aids, not proofs.

This companion supports the practical use of:

Appendix G, Appendix Y, Appendix Z.

It provides example schemas for:

run.yaml, artifact_manifest.json, eval.json, ledger.json, diagnostics.json, gate.json, proof_store_entry.json, reproducibility_manifest.json.

No companion template, execution package, replay record, audit checklist, stored object, or reproducibility manifest may replace:

$$B\text{-Gate}^{+}, \quad \mathrm{Eval}_{n,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}).$$

Remark 2.1 (Companion status)**.** This document is operational and illustrative. It may define recommended schemas and checklists. It does not certify any proof object by itself.

Remark 2.2 (Relation to Appendix Z)**.** Appendix Z defines the execution and reproducibility schema formally. This companion provides practical templates for instantiating that schema. When in doubt, Appendix Z takes precedence.

Remark 2.3 (Final companion boundary)**.** This is the final operational companion. Its purpose is to make the AK infrastructure executable and auditable without confusing execution, replay, storage, or audit with proof.

2.2 C2.1bis. v20 execution hardening and atomic status boundary

Definition 2.4 (Execution-sovereignty boundary)**.** The execution-sovereignty boundary is the rule that successful execution, replay, storage, hash validation, manifest completion, audit-pass status, or reproducibility level does not change a Core, interface, or bridge verdict unless the output is converted into the declared verifier-side record and passes the applicable verifier-side protocol.

Definition 2.5 (Execution-to-verifier conversion record)**.** An execution-to-verifier conversion record declares the execution output, target verifier-side schema, scope, environment, artifact identities, authority, hypotheses, result status $pass/reject/undefined$, and typed failure route. It is required whenever an execution artifact, replay result, proof-store object, or audit record is consumed as evidence.

Definition 2.6 (Replay atomic-status mapping)**.** Replay results map to atomic statuses only as follows: replay-pass may support pass for the declared replay-support criterion only; replay-fail and replay-divergent map to reject for that replay-support criterion; replay-incomplete and replay-blocked map to undefined; and replay-not-applicable maps to not_invoked only under a declared scope-exclusion policy, otherwise to undefined for theorem-facing use. No replay result by itself proves theorem validity.

Definition 2.7 (Execution defect typing)**.** Execution adequacy, environment compatibility, artifact identity, replay divergence, and proof-store status are normally status-ledger or qualitative-governance components. They are not metric-admissible unless a separate support record converts them into a bottleneck discrepancy bound under the Appendix M ledger catalog.

Proposition 2.8 (Replay and audit are support checks, not theorem checks by default)**.** *A replay-pass or audit-pass result verifies only the declared execution or audit-support criterion. It does not prove the mathematical claim interpreted from that execution unless the required verifier-side proof or conversion record is present.*

Proof. Replay checks reconstructibility under a declared environment and equivalence criterion. Audit checks completeness and consistency of records. Theorem validity requires the corresponding mathematical proof, diagnostic, gate, ledger, and claim-use obligations. □

Remark 2.9 (Execution status-token discipline)**.** Template statuses such as verified, audit-passed, replay-passed, complete, and adequate are workflow statuses unless explicitly mapped to Chapter 7 atomic statuses by a scoped conversion record. They do not replace Appendix U agent-output labels, Appendix Y proof-store statuses, Appendix Z replay statuses, Chapter 1 claim roles, or CR-v20 baseline invocation governance.

2.3 C2.1ter. v20 theorem-facing execution boundary and final-CR deferral

Definition 2.10 (Execution status-axis record)**.** An *execution status-axis record* keeps document role, evidence status, invocation role, conformance status, execution lifecycle, replay grade, storage status, and verifier atomic status in separate fields. No field is inferred by copying another.

Definition 2.11 (Theorem-facing execution entry)**.** A *theorem-facing execution entry* extends a generic execution package with immutable content identity, artifact role, source and authority binding, schema and environment identity, dependency closure, claim link, status axes, adequacy preflight, semantic-preservation record, permitted use, forbidden stronger reading, typed failure routes, repair ancestry, replay links, and separate verification, approval, promotion, and publication authorities. A generic execution record cannot be consumed as theorem, bridge, recovery, Return, or release evidence without this extension.

Definition 2.12 (Execution final-CR deferral record)**.** An *execution final-CR deferral record* states that Companion execution artifacts remain local support records until Companion cross-audit and global dependency closure are frozen. A generated Claim UID, release row, promotion receipt, or registry file is noncanonical unless issued by the final CR-v20 process.

Proposition 2.13 (Execution completion is non-promotional)**.** *Execution completion, operational replay, semantic replay, audit pass, or package publication does not automatically promote the underlying mathematical claim.*

Proof. Each listed event verifies a different support predicate. Mathematical promotion additionally requires theorem evidence and authorized claim governance. □

2.4 C2.2. Execution package overview

Definition 2.14 (Execution package)**.** An *execution package* is a collection of files and records sufficient to reconstruct a declared execution run. A recommended execution package contains:

(i) run.yaml;

(ii) artifact_manifest.json;

(iii) eval.json, if repaired evaluation is computed or checked;

(iv) ledger.json, if a budget check is used;

(v) diagnostics.json, if diagnostics are computed;

(vi) gate.json, if a Core gate is checked;

(vii) proof_store_entry.json;

(viii) reproducibility_manifest.json;

(ix) logs;

(x) environment records;

(xi) referenced artifacts.

Definition 2.15 (Execution package status)**.** An execution package has one of the following statuses:

(i) draft;

(ii) incomplete;

(iii) replay-ready;

(iv) replay-passed;

(v) replay-failed;

(vi) audit-ready;

(vii) audit-passed;

(viii) audit-failed;

(ix) blocked;

(x) superseded;

(xi) rejected.

Remark 2.16 (Package status is not theorem status)**.** An execution package may be replay-passed or audit-passed while the underlying mathematical claim remains unproved. The package status and claim status must remain distinct.

2.5 C2.3. run.yaml template

Definition 2.17 (run.yaml operational template)**.** The run.yaml template records the scope, inputs, parameters, commands, environment, outputs, verification criteria, and replay instructions of an execution run.

A recommended template is:

```
run:
  id: "run-0001"
  title: "AK execution run"
  scope:
    type: "persistence_compute / repaired_evaluation / representative_check / tower_artifact_check /
          diagnostics / gate_check / ledger_check / formal_verify / replay / audit"
    description: "declared run scope"
  status: "draft / replay-ready / replay-passed / replay-failed / audit-ready / audit-passed / blocked"

inputs:
  artifacts:
    - id: "artifact-input-001"
      role: "input"
      path_or_uri: "path/to/input"
      hash: "sha256:..."
      required: true

parameters:
  field: "k"
  window_policy:
    windows:
      - id: "W1"
        interval: "[a,a')"
        status: "declared"
  threshold_policy:
    tau: "declared_value"
    endpoint_policy: "declared"
  monitored_degrees:
    - 0
    - 1
  random_seed:
    used: false
    value: null

core_repaired_objects:
  repaired_evaluation:
    used: true
    expression: "Eval_{i,W,tau}(F) = T_bar_tau(W_W P_i(F))"
    eval_record: "eval.json"
  admissible_representative:
    used: false
    expression: "C_{tau,W}F"
    representative_record: "artifact-or-record-id"
  tower_artifact:
    used: false
    expression: "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(F_infty)"
    artifact_record: "artifact-or-record-id"

environment:
  environment_id: "env-0001"
  os: "declared"
  container: "image_digest_or_none"
  language_versions:
    python: "version_or_none"
    coq: "version_or_none"
    lean: "version_or_none"
```

```
  dependencies:
    lockfile: "path/to/lockfile"
    hash: "sha256:..."

commands:
  - id: "cmd-001"
    description: "command description"
    command: "exact command or workflow step"
    expected_output: "declared"

outputs:
  artifacts:
    - id: "artifact-output-001"
      role: "output"
      path_or_uri: "path/to/output"
      hash_policy: "required / optional / none"

verification:
  criteria:
    - "declared criterion"
  authority: "declared tool or agent"
  expected_result: "pass / reject / undefined"

replay:
  instructions: "steps to replay"
  equivalence_criterion: "exact_hash / numerical_tolerance / proof_assistant_acceptance /
       diagnostic_equality / semantic"
  tolerance: null

non_claims:
  - "Successful execution is not proof by itself."
  - "Replay pass is not theorem validity."
  - "Execution artifacts do not replace Core verifier conditions."
```

Remark 2.18 (Run scope must be narrow)**.** A run should have a clear scope. A run that computes, verifies, audits, and interprets theorem-level meaning without separation is difficult to audit.

2.6 C2.3bis. v20 run-scope, detector, higher-edge, and Return fields

Definition 2.19 (v20 run-scope extension)**.** A theorem-facing run additionally fixes the source class, monitored degree set, invoked higher degree n, right-open windows, threshold and endpoint policies, exact comparison mode, detector source stage, family and coverage scope, representative policy, actual tower-morphism scope, transient-defect scope, finite-to-infinite target, Return predicate, Known-Theorem Recovery track, and immutable scope-exclusion policy. Blank fields are undefined; only an immutable exclusion policy permits not_invoked.

Proposition 2.20 (Run configuration does not authorize a theorem)**.** *Declaring a detector stage, higher edge, Type IV scope, reduction, or Return field in* run.yaml *does not prove that the declared theorem or artifact exists.*

Proof. A configuration selects intended inputs and checks. Existence, applicability, completeness, and mathematical truth require separate evidence. □

2.7 C2.4. artifact_manifest.json template

Definition 2.21 (artifact_manifest.json operational template)**.** The artifact_manifest.json template records all artifacts used or produced by an execution run.

A recommended template is:

```
{
  "artifact_manifest": {
    "id": "artifact-manifest-0001",
    "associated_run": "run-0001",
    "status": "draft / complete / incomplete / audit-ready / superseded",
    "artifacts": [
      {
        "id": "artifact-001",
        "role": "input / output / intermediate / log / reference",
        "type": "data / code / proof_script / barcode / eval / representative / tower_artifact / diagram /
            ledger / diagnostic / gate / manifest / log",
        "path_or_uri": "path/to/artifact",
        "version": "v1",
        "hash": "sha256:...",
        "hash_required": true,
        "producer_run": "run-0000",
        "consumer_runs": ["run-0001"],
        "supports": [
          "Eval_{i,W,tau}(F) / C_{tau,W}F / phi_{i,W,tau} / ledger / diagnostics / gate /
              proof_store_entry"
        ],
        "adequacy_status": "unchecked / adequate / inadequate / pending",
        "audit_notes": []
      }
    ],
    "non_claims": [
      "Artifact availability is not artifact adequacy.",
      "Artifact manifest completeness is not proof."
    ]
  }
}
```

Remark 2.22 (Adequacy must be checked)**.** The manifest records existence and identity. A separate adequacy check is required to confirm whether the artifact supports the cited claim or computation.

2.8 C2.5. eval.json template

Definition 2.23 (eval.json operational template)**.** The eval.json template records repaired evaluation computations:

$$\mathrm{Eval}_{i,W,\tau}(F) = T_{\tau}^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big).$$

A recommended template is:

```
{
  "evaluation": {
    "id": "eval-0001",
    "associated_run": "run-0001",
    "input_object": "F",
    "degree": 1,
    "window": {
      "id": "W1",
      "interval": "[a,a')",
      "policy": "declared"
    },
    "threshold": {
      "tau": "declared_value",
      "endpoint_policy": "declared"
    },
    "persistence_extraction": {
      "method": "declared",
      "target": "Perscons_k",
```

```
      "constructibility_status": "pass / reject / undefined"
    },
    "repaired_evaluation": {
      "expression": "Eval_{i,W,tau}(F) = T_bar_tau(W_W P_i(F))",
      "barcode_policy": "declared",
      "window_restriction_policy": "declared",
      "threshold_deletion_policy": "delete bars <= tau",
      "result_profile": "artifact-or-description",
      "vanishing_result": "zero / nonzero / undefined / rejected"
    },
    "artifacts": ["artifact-001"],
    "verification": {
      "status": "candidate / computed / verified / rejected / undefined",
      "authority": "declared"
    },
    "non_claims": [
      "The existence of eval.json does not imply Eval correctness.",
      "Eval summary is not audit-sufficient without reconstructible data."
    ]
  }
}
```

Remark 2.24 (Evaluation record is now explicit)**.** This template is the operational replacement for older exact-collapse execution wording. Execution records should refer to $\mathrm{Eval}_{i,W,\tau}$, not to a global exact collapse operation.

2.9 C2.5bis. Detector and higher-edge execution records

Definition 2.25 (Detector execution record)**.** A *detector execution record* binds one immutable source artifact and exactly one source stage among

$$\text{windowed\_prethreshold}, \quad \text{repaired\_postthreshold}, \quad \text{kernel\_defect}, \quad \text{cokernel\_defect}.$$

It records certificate status separately from mathematical value, together with degree, window, threshold, endpoint convention, family membership, soundness, zero completeness, coverage, non-adaptivity or holdout policy, excess-bound source, computation identity, and typed failure routes. The standard v20 finite detector is sourced at windowed_prethreshold; another stage requires its own theorem or explicit stage transport.

Definition 2.26 (Higher-edge execution record)**.** A *higher-edge execution record* stores the full standard degree-n eligibility, realization, extractor, $E_n(0) = 0$, H^{-n}-identification, naturality, change-compatibility, artifact, and non-circularity package. The default direction is repaired zero to Ext^n-zero; reverse use requires a separately bound zero-reflection theorem.

Proposition 2.27 (Zero-looking output is not certified zero)**.** *An empty array, zero scalar, no-hit result, green dashboard, or successful process exit is not a certified zero detector or higher-edge discharge when the corresponding certificate is incomplete.*

Proof. The mathematical value and certificate status are independent fields. A zero value without applicability and completeness evidence leaves the theorem-facing result undefined. □

2.10 C2.6. ledger.json template

Definition 2.28 (ledger.json operational template)**.** The ledger.json template records ledger components, aggregation semantics, gap values, and budget verdict states.

A recommended template is:

```
{
  "ledger": {
    "id": "ledger-0001",
    "associated_run": "run-0001",
    "ledger_semantics": {
      "type": "additive / max / weighted / product / hybrid",
      "identity": "0 or declared",
      "strict_order": "<",
      "aggregation_rule": "declared"
    },
    "components": [
      {
        "id": "delta-001",
        "symbol": "delta_disc",
        "source": "discretization / measurement / normalization / bridge / categorical_comparison /
            arithmetic_transfer / tower_transfer",
        "value": "declared numeric or symbolic bound",
        "unit": "declared_or_none",
        "artifact_reference": "artifact-001",
        "status": "declared / computed / bounded / undefined"
      }
    ],
    "aggregate": {
      "val_B_d_met": "computed_or_declared",
      "gap_tau": "declared",
      "comparison": "val_B(d_met) < gap_tau(B_family)",
      "result": "pass / reject / undefined / not_invoked"
    },
    "audit": {
      "replayable": true,
      "notes": []
    },
    "non_claims": [
      "The existence of ledger.json does not imply budget pass.",
      "Budget pass does not replace B-Gate+ or diagnostics."
    ]
  }
}
```

Remark 2.29 (Strict comparison)**.** The strict inequality

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B})$$

must be recorded as strict. A non-strict or ambiguous comparison should not be accepted as an Audit Mode pass.

2.11 C2.6bis. Selected-sub-DAG quantale ledger execution

Definition 2.30 (Selected execution route)**.** A *selected execution route* P is the finite necessary target sub-DAG containing every branch consumed by the proof object. It is not required to be a single linear path.

Definition 2.31 (Execution scalarization record)**.** The active metric value system is a declared commutative quantale $(\mathcal{V}, \oplus, 0_{\mathcal{V}}, \preceq)$ with a monotone subadditive scalarization

$$\mathrm{val}_B : \mathcal{V} \longrightarrow [0, +\infty], \qquad \mathrm{val}_B(0_{\mathcal{V}}) = 0.$$

For the finite route P, aggregation uses only its finite ordered commutative-monoid reduct:

$$\mathfrak{d}_{C2}(P) = \bigoplus_{e \in S_P} \delta_e.$$

Each primitive component supplies actual pre-threshold profiles with

$$d_B\left(B_e^{\mathrm{src}}, B_e^{\mathrm{tgt}}\right) \leq \mathrm{val}_B(\delta_e),$$

and a relation alias, refinement, aggregate, or disjoint to every overlapping namespace.

Proposition 2.32 (Execution records do not multiply discrepancy)**.** *Mirrors, retries, logs, aliases, manifests, proof-store entries, DAG edges, and replay records do not create additional independent metric discrepancy.*

Proof. They are representations or operational events concerning the same primitive comparison unless a separate theorem proves a disjoint sequential contribution. □

Proposition 2.33 (Qualitative execution failure is non-scalar)**.** *Missing source authority, schema permission, artifact adequacy, semantic identity, environment compatibility, detector completeness, reduction, Return, or replay interpretation is not repaired by metric slack.*

Proof. The failures are theorem, authority, identity, or governance obligations, not bottleneck discrepancies. □

2.12 C2.7. diagnostics.json template

Definition 2.34 (diagnostics.json operational template)**.** The diagnostics.json template records diagnostic computations, especially monitored Type IV diagnostics.

A recommended template is:

```
{
  "diagnostics": {
    "id": "diagnostics-0001",
    "associated_run": "run-0001",
    "diagnostic_type": "TypeIV / PH / Ext / tower / advisory",
    "monitored_degrees": [0, 1],
    "windows": [
      {
        "id": "W1",
        "interval": "[a,a')"
      }
    ],
    "thresholds": [
      {
        "tau": "declared_value"
      }
    ],
    "tower_comparisons": [
      {
        "id": "tower-comp-001",
        "degree": 1,
        "window": "W1",
        "threshold": "tau",
        "tower_artifact": "phi_{i,W,tau}: ColimProfile_{i,W,tau}(F_bullet) -> ApexProfile_{i,W,tau}(
            F_infty)",
        "artifact_status": "candidate / declared / verified / rejected / undefined",
        "source_profile": "ColimProfile_{i,W,tau}(F_bullet)",
        "target_profile": "ApexProfile_{i,W,tau}(F_infty)",
        "kernel_defect_object": "artifact-or-description",
        "cokernel_defect_object": "artifact-or-description",
        "terminal_generic_defect_dimensions": {
          "mu_component": "value_or_undefined",
          "nu_component": "value_or_undefined"
        },
```

```
        "verdict": "certified_zero / obstructed / undefined / not_invoked"
      }
    ],
    "summary": {
      "mu_Collapse": "value_or_undefined",
      "nu_Collapse": "value_or_undefined",
      "diagnostic_verdict": "certified_zero / obstructed / undefined / not_invoked"
    },
    "artifact_references": [],
    "non_claims": [
      "Diagnostic summary is insufficient without reconstructible computation.",
      "A Type IV proxy is not monitored Type IV diagnosis.",
      "A monitored diagnostic requires a declared phi_{i,W,tau} artifact."
    ]
  }
}
```

Remark 2.35 (Tower artifact must be reconstructible)**.** For monitored Type IV diagnostics, the tower comparison artifact

$$\phi_{i,W,\tau}$$

and its defect objects must be reconstructible. A summary value alone is not audit-sufficient. The execution template must not use a zero pair as a substitute for the full typed Type IV diagnostic record.

2.13 C2.7bis. Type IV, transient, and apex-agreement execution records

Definition 2.36 (v20 Type IV execution record)**.** A *v20 Type IV execution record* binds the actual morphism

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty),$$

its source and target semantic authority, transition and colimit construction, apex artifact, apex-agreement status, terminal-tameness evidence, full kernel and cokernel profiles, degreewise $\mu_{i,W,\tau}$ and $u_{i,W,\tau}$, and the aggregate over one fixed (W, τ) and finite monitored degree set. Its specialized status is not_invoked, undefined, certified_zero, or obstructed; the first two carry no numerical pair.

Definition 2.37 (Transient and full-defect execution record)**.** A *transient and full-defect execution record* keeps terminal Type IV, full-interior defect, long-transient defect, full kernel/cokernel vanishing, morphism isomorphism, and apex agreement as separate fields and separate evidence obligations.

Proposition 2.38 (Terminal diagnostics do not close stronger fields)**.** *A terminal certified-zero record does not automatically set transient, full-defect, isomorphism, apex-agreement, or external information-preservation fields to pass.*

Proof. The fields concern distinct source profiles and theorem strengths. No execution rule may copy one verdict into another. □

2.14 C2.8. gate.json template

Definition 2.39 (gate.json operational template)**.** The gate.json template records Core gate checks and clause-level verdicts.

A recommended template is:

```
{
  "gate": {
    "id": "gate-0001",
    "associated_run": "run-0001",
    "gate_type": "B-Gate+ / CollapseAdmissible / interface_pre_gate / other",
    "input_object": "object-id",
```

```
    "window": "W1",
    "threshold": "tau",
    "clauses": [
      {
        "id": "clause-001",
        "statement": "Eval_{n,W,tau}(F) = 0",
        "result": "pass / reject / undefined",
        "evidence": ["eval-0001"]
      },
      {
        "id": "clause-002",
        "statement": "Ext^n = 0 in eligible regime using C_{tau,W}F",
        "result": "pass / reject / undefined / not_invoked",
        "evidence": ["artifact-002"]
      },
      {
        "id": "clause-003",
        "statement": "Type IV diagnostics clean using phi_{i,W,tau}",
        "result": "pass / reject / undefined",
        "evidence": ["diagnostics-0001"]
      },
      {
        "id": "clause-004",
        "statement": "budget condition val_B(d_met) < gap_tau(B_family)",
        "result": "pass / reject / undefined",
        "evidence": ["ledger-0001"]
      }
    ],
    "final_verdict": "pass / reject / undefined / blocked",
    "verifying_authority": "declared",
    "non_claims": [
      "The existence of gate.json does not imply gate pass.",
      "Clause-level support is required."
    ]
  }
}
```

Remark 2.40 (Gate pass must not be asserted globally)**.** A gate pass is scoped to the declared object, window, threshold, and clauses. It should not be generalized without a separate theorem.

2.15 C2.9. proof_store_entry.json template

Definition 2.41 (proof_store_entry.json operational template)**.** The proof_store_entry.json template records an object stored in the Proof Store and its status metadata.

A recommended template is:

```
{
  "proof_store_entry": {
    "id": "pse-0001",
    "object_type": "search_artifact / candidate / formal_fragment / verified_fragment /
        certificate_candidate / core_certificate / core_facing_object / bridge_program / bridge_theorem /
         non_claim",
    "title": "declared title",
    "content_reference": "path_or_uri_or_hash",
    "version": "v1",
    "status": "draft / candidate / verified / rejected / superseded / [Spec]",
    "claim_type": "Core theorem / Core lemma / operational policy / interface theorem / bridge candidate /
        bridge theorem / non-claim",
    "core_facing_payload": {
      "Eval": "Eval_{i,W,tau}(F) / none / pending",
      "representative": "C_{tau,W}F / none / pending",
```

```
      "tower_artifact": "phi_{i,W,tau} / none / pending",
      "diagnostics": "mu_Collapse, nu_Collapse / none / pending",
      "ledger": "val_B(d_met) < gap_tau(B_family) / none / pending"
    },
    "dependencies": [
      {
        "id": "dep-001",
        "type": "logical / artifact / ledger / diagnostic / bridge / advisory",
        "target": "pse-or-artifact-id",
        "status": "discharged / pending / failed / advisory"
      }
    ],
    "artifacts": ["artifact-001"],
    "verification_records": ["verification-001"],
    "audit_records": ["audit-001"],
    "provenance": {
      "created_by": "agent-or-human",
      "created_at": "timestamp-or-version",
      "modified_by": [],
      "status_history": []
    },
    "non_claims": [
      "Storage is not certification.",
      "Proof Store entry does not imply theorem status."
    ]
  }
}
```

Remark 2.42 (Storage status versus claim status)**.** The object may be stored successfully while the mathematical claim remains unverified. Storage and claim status must remain separated.

2.16 C2.9bis. Proof-store identity, authority, and permitted use

Definition 2.43 (Proof-store authority separation)**.** Read, write, verification, audit, approval, promotion, demotion, and publication authorities are independent. Co-location in one UI, agent, service account, or database does not compose them.

Definition 2.44 (Claim-relative semantic authority)**.** Semantic authority is claim-relative. An artifact may be authoritative for one statement, degree, benchmark clause, coefficient passage, colimit construction, or apex interpretation without being authoritative for a stronger or adjacent claim. Basename, path, upload time, display title, query rank, storage location, and mirror count are navigation metadata rather than mathematical identity.

Proposition 2.45 (Storage and authority do not self-compose)**.** *Storing an artifact under an authoritative account or in an approved proof store does not make its content theorem evidence.*

Proof. Storage authority and mathematical verification authority are separate predicates. The theorem-facing entry and claim-use preflight remain required. □

2.17 C2.10. Reproducibility manifest template

Definition 2.46 (Reproducibility-manifest operational template)**.** The file reproducibility_manifest.json links the run schema, artifacts, and environment. It also links evaluation and ledger records. Diagnostic, gate, replay, and audit records are linked separately.

A recommended template is:

```
{
  "reproducibility_manifest": {
```

```
    "id": "repro-manifest-0001",
    "associated_run": "run-0001",
    "run_yaml": "run.yaml",
    "artifact_manifest": "artifact_manifest.json",
    "environment": {
      "environment_id": "env-0001",
      "reference": "environment.lock / container_digest / proof_assistant_env"
    },
    "records": {
      "eval": "eval.json / not_applicable",
      "ledger": "ledger.json / not_applicable",
      "diagnostics": "diagnostics.json / not_applicable",
      "gate": "gate.json / not_applicable",
      "proof_store_entry": "proof_store_entry.json"
    },
    "replay": {
      "result": "replay-pass / replay-fail / replay-incomplete / replay-divergent / replay-not-applicable /
          replay-blocked",
      "equivalence_criterion": "exact_hash / tolerance / formal_acceptance / diagnostic_equality / semantic
          ",
      "replay_log": "path/to/replay.log"
    },
    "reproducibility_level": "R0 / R1 / R2 / R3 / R4 / R5",
    "audit": {
      "result": "audit-pass / audit-fail / audit-incomplete / audit-blocked",
      "scope": "declared",
      "audit_record": "audit-001"
    },
    "non_claims": [
      "Reproducibility manifest is not proof.",
      "Replay pass is not theorem validity."
    ]
  }
}
```

Remark 2.47 (Top-level connector)**.** The reproducibility manifest is the top-level connector for execution audit. It should not duplicate all records, but it should link them unambiguously.

2.18 C2.10bis. Replay hierarchy and Known-Theorem Recovery quarantine

Definition 2.48 (Operational, semantic, and mathematical replay)**.** An *operational replay* reconstructs declared bytes, commands, computations, or formal checks inside a pinned boundary. A *semantic replay* additionally verifies preservation of statements, hypotheses, scopes, source authority, detector stages, comparison modes, status axes, and interpretation arrows. A *mathematical reconstruction* independently re-establishes the theorem-level derivation. In general,

$$\text{operational replay} \Rightarrow \text{semantic replay} \Rightarrow \text{mathematical reconstruction}.$$

Definition 2.49 (Recovery execution quarantine record)**.** For a Known-Theorem Recovery run, a *recovery execution quarantine record* proves by input and dependency inspection that the benchmark proof, known truth value, target constant, exceptional bound, conclusion-derived witness, and equivalent oracle did not enter realization, certificate, reduction, or Return-proof construction. The immutable target statement and locator may be present only as target specification, Return consequent, and final validation reference.

Proposition 2.50 (Replay cannot cure circular recovery)**.** *Reproducible regeneration of a benchmark-contaminated recovery does not make the recovery independent.*

Proof. Replay preserves the contaminated dependency graph. It does not remove the conclusion-side input that caused circularity. □

2.19 C2.11. Verification record template

Definition 2.51 (Verification record template)**.** A *verification record template* records the result of a scoped verification action.

A recommended template is:

```
{
  "verification_record": {
    "id": "verification-001",
    "object": "pse-0001",
    "criteria": [
      "declared criterion 1",
      "declared criterion 2"
    ],
    "authority": {
      "type": "human / AI / proof_assistant / computation_engine / audit_tool",
      "name": "declared",
      "scope": "declared"
    },
    "environment": "env-0001",
    "result": "pass / fail / inconclusive / blocked / not_applicable",
    "evidence": ["artifact-001"],
    "logs": ["verification.log"],
    "core_conditions_checked": {
      "Eval_{i,W,tau}": "yes / no / not_applicable",
      "C_{tau,W}F": "yes / no / not_applicable",
      "phi_{i,W,tau}": "yes / no / not_applicable",
      "diagnostics": "yes / no / not_applicable",
      "ledger": "yes / no / not_applicable",
      "gate": "yes / no / not_applicable"
    },
    "non_claims": [
      "Verification result is scoped to declared criteria.",
      "Verification pass does not imply unrelated claims."
    ]
  }
}
```

2.20 C2.12. Audit record template

Definition 2.52 (Audit record template)**.** An *audit record template* records whether a proof object, certificate, or execution package is reconstructible within a declared scope.

A recommended template is:

```
{
  "audit_record": {
    "id": "audit-001",
    "scope": {
      "objects": ["pse-0001"],
      "runs": ["run-0001"],
      "artifacts": ["artifact-001"],
      "records": ["eval.json", "ledger.json", "diagnostics.json", "gate.json"]
    },
    "checks": [
      {
        "id": "audit-check-001",
        "description": "artifact hash consistency",
        "result": "pass / reject / undefined"
      },
      {
        "id": "audit-check-002",
```

```
        "description": "manifest completeness",
        "result": "pass / reject / undefined"
      },
      {
        "id": "audit-check-003",
        "description": "replayability",
        "result": "pass / reject / undefined"
      },
      {
        "id": "audit-check-004",
        "description": "Core-facing record reconstructibility",
        "result": "pass / reject / undefined"
      }
    ],
    "result": "audit-pass / audit-fail / audit-incomplete / audit-blocked",
    "notes": [],
    "non_claims": [
      "Audit pass is not mathematical proof by itself.",
      "Audit checks reconstructibility and record integrity."
    ]
  }
}
```


2.21 C2.13. R0–R5 reproducibility checklist

Definition 2.53 (Reproducibility checklist)**.** A *reproducibility checklist* is an operational checklist for assigning reproducibility level R0–R5.

Level R0 checklist.

- Result is stated.
- Execution schema is missing or insufficient.
- Required artifacts are missing or unavailable.
- Replay is not possible.

Level R1 checklist.

- Artifacts are referenced.
- Some metadata exists.
- Run cannot be replayed from available records.

Level R2 checklist.

- Inputs are recorded.
- Parameters are partially or fully recorded.
- Outputs are recorded.
- Environment or commands are incomplete.

Level R3 checklist.

- Inputs, outputs, parameters, and commands are recorded.
- Compatible environment is available.
- Replay succeeds under declared tolerance or equivalence.

Level R4 checklist.

- Captured environment is available.
- Artifact hashes match.
- Exact or content-identical replay succeeds.
- Logs and outputs are stable.

Level R5 checklist.

- R4 conditions hold.
- Independent audit is recorded.
- Proof Store entries are linked.
- Repaired evaluation records are linked where applicable.
- Ledger, diagnostics, and gate records are linked where applicable.
- Manifest completeness is checked.
- Audit result is audit-pass.

Remark 2.54 (R5 is not theorem validity)**.** R5 is a high reproducibility and audit level. It is still not theorem validity unless the proof content is valid.

2.22 C2.13bis. Vector-valued replay maturity

Definition 2.55 (Replay maturity vector)**.** The replay maturity of a package is recorded as a vector

$$(R_{\mathrm{op}}, R_{\mathrm{sem}}, R_{\mathrm{math}}, R_{\mathrm{src}}, R_{\mathrm{env}}, R_{\mathrm{audit}}),$$

whose components respectively describe operational reproducibility, semantic preservation, mathematical reconstruction, source rebinding, environment reconstruction, and independent audit. The legacy scalar R0–R5 label is retained only as a coarse operational summary and does not dominate the vector components.

Proposition 2.56 (R5 does not imply semantic or mathematical completeness)**.** *A package may satisfy the legacy R5 checklist while lacking source rebinding, semantic preservation, interpretation, or independent mathematical reconstruction.*

Proof. The scalar checklist aggregates operational support dimensions and does not prove every component of the replay maturity vector. □

2.23 C2.14. Replay procedure example

A recommended replay procedure is:

(R1) Retrieve reproducibility_manifest.json.

(R2) Retrieve run.yaml.

(R3) Retrieve artifact_manifest.json.

(R4) Check required input artifact availability.

(R5) Verify hashes for all hash-bound artifacts.

(R6) Reconstruct the declared environment.

(R7) Execute commands in declared order.

(R8) Compare outputs under the declared replay equivalence criterion.

(R9) Recompute or inspect eval.json, if applicable.

(R10) Recompute or inspect ledger.json, if applicable.

(R11) Recompute or inspect diagnostics.json, if applicable.

(R12) Recompute or inspect gate.json, if applicable.

(R13) Record replay result.

(R14) Update reproducibility level if appropriate.

(R15) If replay fails, create a reproducibility failure record.

Remark 2.57 (Replay should not update claim status automatically)**.** Replay pass may support audit status, but it should not automatically promote the mathematical claim. Promotion requires the relevant evidence and authority.

2.24 C2.15. Failure and demotion examples

Definition 2.58 (Execution failure example)**.** An *execution failure example* is an operational example showing when an execution package should be reviewed, demoted, or rejected.

Common examples include:

(E1) *Missing artifact.* A required input artifact is absent. Recommended status: replay-blocked.

(E2) *Hash mismatch.* A retrieved artifact does not match its declared hash. Recommended status: audit-blocked until resolved.

(E3) *Environment unavailable.* The required environment cannot be reconstructed. Recommended status: R2 or lower.

(E4) *Parameter ambiguity.* Threshold τ, window W, monitored degree i, or ledger semantics are missing. Recommended status: incomplete.

(E5) *Repaired evaluation inconsistency.* eval.json states

$$\mathrm{Eval}_{i,W,\tau}(F) = 0$$

but the barcode, window, threshold, or deletion policy is missing. Recommended status: evaluation-incomplete.

(E6) *Representative ambiguity.* $C_{\tau,W}F$ is invoked, but no admissible representative record is attached. Recommended status: representative-incomplete.

(E7) *Ledger inconsistency.* $\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}})$ in $\mathrm{ledger.json}$ does not match component aggregation. Recommended status: verification-fail or audit-fail.

(E8) *Diagnostic summary without computation.* μ_{Collapse} and u_{Collapse} are stated, but $\phi_{i,W,\tau}$ or defect objects are missing. Recommended status: diagnostic-incomplete.

(E9) *Gate verdict without clauses.* $\mathrm{gate.json}$ states pass, but clause-level evidence is missing. Recommended status: gate-incomplete.

(E10) *Replay divergence.* Replayed outputs fail the declared equivalence criterion. Recommended status: replay-fail.

(E11) *AI-generated proof object without status label.* Recommended status: non-verdict until labeled and reviewed.

(E12) *Stored object promoted without evidence.* Recommended status: demote to previous supported status.

Remark 2.59 (Demotion protects integrity)**.** Demotion is not punitive. It preserves the reliability of the proof infrastructure.

2.25 C2.16. Minimal execution package manifest

Definition 2.60 (Minimal execution package manifest)**.** A *Minimal Execution Package Manifest* is the minimum operational record needed to audit an execution package. It contains:

(i) execution package identifier;

(ii) run identifier;

(iii) run scope;

(iv) $\mathrm{run.yaml}$ reference;

(v) artifact manifest reference;

(vi) environment reference;

(vii) repaired evaluation reference, if applicable;

(viii) ledger reference, if applicable;

(ix) diagnostics reference, if applicable;

(x) gate reference, if applicable;

(xi) proof-store entry reference;

(xii) replay result;

(xiii) reproducibility level;

(xiv) audit result;

(xv) non-claim list.

A compact schematic manifest is:

```
minimal_execution_package_manifest:
  id: "exec-package-0001"
  status: "draft / replay-ready / replay-passed / audit-ready / audit-passed / failed"
  run: "run-0001"
  run_scope: "declared"
  run_yaml: "run.yaml"
  artifact_manifest: "artifact_manifest.json"
  environment: "env-0001"
  records:
    eval: "eval.json / not_applicable"
    ledger: "ledger.json / not_applicable"
    diagnostics: "diagnostics.json / not_applicable"
    gate: "gate.json / not_applicable"
    proof_store_entry: "proof_store_entry.json"
  replay:
    result: "replay-pass / replay-fail / replay-incomplete / replay-blocked"
    level: "R0 / R1 / R2 / R3 / R4 / R5"
  audit:
    result: "audit-pass / audit-fail / audit-incomplete / audit-blocked"
  non_claims:
    - "Execution package is not proof by itself."
    - "Reproducibility supports audit but does not prove theorem validity."
```

Remark 2.61 (Operational manifest versus Core manifest)**.** The Minimal Execution Package Manifest does not replace the Core Minimal Manifest of Appendix G. It supports execution audit. Core certification still requires verifier-side Core data.

2.26 C2.17. Recommended execution workflow

A recommended execution and reproducibility workflow is:

(X1) Define the run scope.

(X2) Create run.yaml.

(X3) Register input artifacts.

(X4) Record environment information.

(X5) Execute the run.

(X6) Store output artifacts.

(X7) Create or update artifact_manifest.json.

(X8) If repaired evaluation is used, create eval.json.

(X9) If budget is used, create ledger.json.

(X10) If diagnostics are used, create diagnostics.json.

(X11) If gates are checked, create gate.json.

(X12) Register the object in the Proof Store.

(X13) Create reproducibility_manifest.json.

(X14) Attempt replay.

(X15) Assign reproducibility level R0–R5.

(X16) Perform audit.

(X17) If failure is detected, demote or block the relevant object.

(X18) If all required evidence is present, prepare for verifier-side certification.

Remark 2.62 (Workflow boundary)**.** This workflow prepares execution and audit support. It does not itself prove the mathematical claim.

2.27 C2.18. Non-claim examples

Recommended non-claim clauses for execution and reproducibility records are:

```
non_claims:
  - "This execution record is not proof by itself."
  - "Successful execution does not imply certificate success."
  - "Replay pass does not imply theorem validity."
  - "Audit pass checks reconstructibility, not mathematical truth by itself."
  - "Storage is not certification."
  - "Hash match proves content identity, not validity."
  - "eval.json existence does not imply Eval correctness."
  - "ledger.json existence does not imply budget pass."
  - "diagnostics.json existence does not imply Type IV cleanliness."
  - "gate.json existence does not imply gate pass."
  - "Formal replay is scoped to the checked formal statement."
  - "Numerical reproducibility does not imply continuum validity."
  - "No execution artifact replaces B-Gate+."
  - "No execution artifact replaces Eval_{n,W,tau}(F)=0."
  - "No execution artifact replaces C_{tau,W}F when representatives are invoked."
  - "No execution artifact replaces phi_{i,W,tau} when Type IV diagnostics are invoked."
  - "No execution artifact replaces monitored Type IV diagnostics."
  - "No execution artifact replaces the budget condition."
```

2.28 C2.19. Summary of Execution / Reproducibility Companion

Proposition 2.63 (Companion 2 execution-reproducibility package)**.** *As an operational companion document, this companion provides the following package.*

(i) *Execution packages organize run schema, artifacts, repaired evaluations, ledgers, diagnostics, gates, proof-store entries, and reproducibility manifests.*

(ii) run.yaml *records run scope, inputs, parameters, Core-facing object configuration, environment, commands, outputs, and replay instructions.*

(iii) artifact_manifest.json *records artifact identity and role but does not prove artifact adequacy.*

(iv) eval.json *records repaired evaluation computations but does not imply evaluation correctness by existence.*

(v) ledger.json *records budget components and comparison but does not imply budget pass by existence.*

(vi) diagnostics.json *records diagnostic computations but does not imply Type IV cleanliness by existence.*

(vii) gate.json *records gate clauses and verdicts but does not imply gate pass by existence.*

(viii) proof_store_entry.json *records stored object metadata but storage is not certification.*

(ix) reproducibility_manifest.json *links execution records for audit.*

(x) *Verification and audit records must be scoped.*

(xi) R0–R5 reproducibility levels classify replay and audit support, not theorem validity.

(xii) Replay procedures should be explicit and should not automatically promote mathematical claim status.

(xiii) Failure cases should trigger review, blocking, or demotion where necessary.

(xiv) Minimal Execution Package Manifest supports execution audit but does not replace the Core Minimal Manifest.

(xv) Non-claim clauses should be included in execution and reproducibility records.

(xvi) Detector certificate status and mathematical value are stored separately at one immutable source stage.

(xvii) Higher-edge and actual-morphism Type IV records are independently typed. Transient-defect, finite-to-infinite, and Return records remain separate.

(xviii) Metric aggregation uses every required branch of a finite selected target sub-DAG and counts each primitive discrepancy once.

(xix) Operational replay, semantic replay, and mathematical reconstruction are distinct.

(xx) Companion closure remains weaker than global dependency closure and final CR-v20 promotion.

Proof. This theorem summarizes operational templates and checklists provided in this companion. Since the companion is non-theorem-bearing with respect to the AK Core, the proof is a structural verification of the companion's execution and reproducibility roles rather than a mathematical theorem about Core validity. □

2.29 C2.20. Explicit exclusions

The following are not asserted in Companion 2.

- No execution package is proof by itself.
- No run.yaml file is proof by itself.
- No artifact manifest proves artifact adequacy.
- No eval.json file proves repaired evaluation correctness by existence.
- No ledger.json file proves budget pass by existence.
- No diagnostics.json file proves Type IV cleanliness by existence.
- No gate.json file proves gate pass by existence.
- No Proof Store entry certifies an object by storage.
- No reproducibility manifest proves theorem validity.
- No replay-pass result proves a theorem by itself.
- No audit-pass result proves mathematical truth by itself.
- No R5 reproducibility level implies theorem validity.
- No hash match proves validity.

- No formal replay proves informal interpretation without an interpretation theorem.
- No numerical reproducibility proves continuum validity.
- No execution artifact replaces B-Gate^{+}.
- No execution artifact replaces the repaired topological condition
$$\mathrm{Eval}_{n,W,\tau}(F) = 0.$$
- No execution artifact replaces admissible representative records
$$C_{\tau,W}F.$$
- No execution artifact replaces tower comparison artifacts
$$\phi_{i,W,\tau}.$$
- No execution artifact replaces monitored Type IV diagnostics.
- No execution artifact replaces the budget condition
$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$
- No Companion document enlarges the Core theorem package.
- No replay-not-applicable result becomes $\mathrm{not\_invoked}$ without a declared scope-exclusion policy.
- No environment, identity, adequacy, replay, or storage defect is compensated by metric slack unless it is separately converted into a metric-admissible component under Appendix M.
- No operational replay is represented as semantic replay or independent mathematical reconstruction.
- No Type IV numerical pair is attached to $\mathrm{not\_invoked}$ or $\mathrm{undefined}$.
- No terminal Type IV zero is copied into transient, full-defect, isomorphism, apex-agreement, or external-preservation fields.
- No linear path that omits a required proof branch is accepted as the selected target sub-DAG.
- No generated Claim UID, promotion receipt, or release row is canonical before the final CR-v20 process.

2.30 C2.21. Final companion closure

Proposition 2.64 (Companion closure principle)**.** *Companion 1 and Companion 2 provide operational calibration, execution, reproducibility, and audit templates for the AK framework. They do not add mathematical theorems, do not modify the Core verifier, do not replace the Problem Interface appendices, and do not promote examples, records, or templates to theorem status.*

Proof. Companion 1 is an arithmetic calibration document. Companion 2 is an execution and reproducibility document. Both documents provide operational templates and non-claim examples. Neither supplies new Core axioms, new Core theorems, new bridge theorems, or theorem-level problem conclusions. Therefore the companion layer is locally and operationally closed but not theorem-expanding, globally dependency-closed, or release-promoted. □

Remark 2.65 (Final operational closure)**.** After this companion is independently reviewed, the v20.0.0 reconstructed document set has a complete construction-stage operational shell: arithmetic calibration is handled by Companion 1, and execution/reproducibility is handled by Companion 2. All theorem-level claims must still pass through the appropriate Core, bridge, interface, and audit disciplines.

2.31 C2.21bis. v20 Companion closure grade and global handoff

Definition 2.66 (Companion closure grade)**.** The positive Companion workflow grades are

$$\text{companion\_local\_complete} \prec \text{companion\_crosschecked} \prec \text{dependency\_closure\_frozen} \prec \text{final\_cr\_promotion\_pending} \prec \text{release\_promoted}.$$

The grade records the greatest completed workflow stage; blocked and undefined override positive progression for the affected target.

Proposition 2.67 (Companion closure is not global release closure)**.** *The state* companion_crosschecked *does not imply frozen global dependency closure, canonical final CR promotion, or release promotion.*

Proof. Those later grades require a cross-document audit and separate governance artifacts not created by the Companion itself. □

Remark 2.68 (Final v20 handoff)**.** After this Companion is independently reviewed and frozen, the next admissible operation is the global cross-audit of Main, Foundation, CM, TB, Technical Extension, Problem Interface, Search / Platform, source-binding and recovery artifacts, and Companion. Only after that closure may the final CR-v20 and release registry be generated.

2.32 C2.22. Provenance and status

This companion depends on:

(i) Appendix G: Minimal Manifest and Proof Object Schema;

(ii) Appendix Y: AI Platform and Proof Store;

(iii) Appendix Z: Execution and Reproducibility Schema;

(iv) Appendix A: Repaired Barcode-Level Collapse and Constructible Persistence;

(v) Appendix B: Admissible Representatives;

(vi) Appendix D: Tower Diagnostics and Type IV Defect Calculus;

(vii) Appendix M: Ledger Semantics and Integrated Defect Catalog;

(viii) Chapter 7: Core Sovereignty and typed audit semantics;

(ix) Chapter 8: Explicit Non-Claims and Core/interface boundary;

(x) Appendix U: Agent Semantics;

(xi) Appendix X: Validity Map and Global Certificate DAG;

(xii) Companion 1: Arithmetic Calibration Companion.

Its role is to support disciplined execution, replay, and audit without converting operational reproducibility into proof.

Status labels for Companion 2.

Operational companion definitions: Definitions 2.14, 2.15, 2.17, 2.21, 2.23, 2.28, 2.34, 2.39, 2.41, 2.46, 2.51, 2.52, 2.53, 2.58, 2.60, 2.4, 2.5, 2.6, 2.7.
Operational companion propositions: Propositions 2.8 and 2.63.
Operational cautions: Remarks 2.1, 2.2, 2.3, 2.16, 2.18, 2.22, 2.24, 2.29, 2.35, 2.40, 2.42, 2.47, 2.54, 2.57, 2.59, 2.61, 2.62, 2.65, 2.9.
v20 strengthened execution definitions: Definitions 2.10, 2.11, 2.12, 2.19, 2.25, 2.26, 2.30, 2.31, 2.36, 2.37, 2.43, 2.44, 2.48, 2.49, 2.55, 2.66.
v20 strengthened execution propositions: Propositions 2.13, 2.20, 2.27, 2.32, 2.33, 2.38, 2.45, 2.50, 2.56, 2.67.
v20 strengthened execution remarks: Remark 2.68.

Explicit exclusions: Section C2.20.
Final companion closure: Proposition 2.64.